

Angular Complete Curriculum — From Zero to Junior-Ready

A complete, self-contained reference covering everything from TypeScript basics through deployment. Designed for someone who knows JavaScript decently and wants to learn Angular in a focused, interview-ready way.

How to use this:

- Read top-to-bottom for the full learning path.
 - Skip to specific topics via the table of contents when you need a refresher.
 - Each topic ends with “Interview Soundbites” — memorize those for interviews.
 - “Try this now” exercises at the end of each topic put the theory into your fingers.
-

Table of Contents

Prerequisites

1. [TypeScript Basics](#)

Core

2. [Angular CLI & Project Structure](#)
3. [Components](#)
4. [Templates & Data Binding](#)
5. [Control Flow \(@if , @for , @switch \)](#)
6. [Directives](#)
7. [Pipes](#)
8. [Component Communication \(@Input , @Output \)](#)
9. [Lifecycle Hooks](#)
10. [Services](#)
11. [Dependency Injection](#)
12. [Standalone Components](#)

Reactivity & Data

13. [Signals](#)
14. [RxJS Basics — Observables, Subjects, BehaviorSubject](#)
15. [RxJS Operators](#)

- 16. [HttpClient](#)
- 17. The [async](#) Pipe

Routing

- 18. [Routing — The Basics](#)
- 19. [Route Params & Query Params](#)
- 20. [Route Guards](#)
- 21. [Lazy Loading](#)

Forms

- 22. [Template-Driven Forms](#)
- 23. [Reactive Forms](#)
- 24. [Form Validation](#)

Advanced

- 25. [Change Detection](#)
- 26. [Content Projection \(<ng-content> \)](#)
- 27. [ViewChild and ContentChild](#)
- 28. [Custom Directives](#)
- 29. [HTTP Interceptors](#)

Production

- 30. [Testing Basics](#)
 - 31. [State Management](#)
 - 32. [Performance Optimization](#)
 - 33. [Deployment & Build Optimization](#)
-

Ground Rules

Before you start, internalize these:

1. **Use Angular 17+ (current is 21).** Many tutorials still teach NgModules and `*ngIf / *ngFor`. The modern way is **standalone components, signals**, and `@if / @for`. Verify any tutorial is from 2024 or later.
2. **Code every day.** Even 30 minutes. Streak beats intensity.
3. **Type the code, don't copy-paste.** Your fingers learn syntax your brain forgets.
4. **Build > watch.** A 1-hour video is worth nothing if you don't build something. Rule of thumb: 1 hour watching = 2 hours building.

5. **Don't feel "ready" before you start applying for jobs.** Imposter syndrome is universal.
-

Topic 1 — TypeScript Basics

Angular is written in TypeScript (TS), not plain JavaScript. TS is just JS with **types** added on top. Your browser can't run TS directly — it gets compiled down to JS.

Why types? Catches bugs before you run the code. If you try to do `"hello" * 2`, TS yells at you immediately. JS would just say `NaN` at runtime.

1. Basic Types

```
let name: string = "Ravi";
let age: number = 25;
let isActive: boolean = true;
let scores: number[] = [90, 85, 78]; // array of numbers
let anything: any = "could be anything"; // avoid 'any' — defeats the purpose
```

The `: string` part is the **type annotation**. You can often skip annotations if TS can figure it out:

```
let name = "Ravi"; // TS infers this is a string
```

2. Functions with Types

```
function add(a: number, b: number): number {
  return a + b;
}
```

- `a: number, b: number` — parameter types
- `: number` after `()` — return type

Optional parameters with `?:`

```
function greet(name: string, title?: string): string {  
  return title ? `${title} ${name}` : name;  
}
```

3. Interfaces

An interface defines the **shape** of an object.

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
  age?: number; // optional  
}  
  
const u: User = {  
  id: 1,  
  name: "Ravi",  
  email: "ravi@example.com"  
};
```

You'll see interfaces everywhere in Angular — for API responses, component inputs, etc.

4. Classes

```
class Person {  
  name: string;  
  age: number;  
  
  constructor(name: string, age: number) {  
    this.name = name;  
    this.age = age;  
  }  
  
  greet(): string {  
    return `Hi, I'm ${this.name}`;  
  }  
}
```

```
}  
}
```

Shorthand — declare and assign properties in the constructor:

```
class Person {  
  constructor(public name: string, public age: number) {}  
}
```

Angular components are classes. You'll write a *lot* of these.

5. Access Modifiers

- `public` — accessible everywhere (default)
- `private` — only inside the class
- `readonly` — can't reassign after init

6. Union Types

```
let id: string | number;  
id = "abc"; // ok  
id = 123;   // ok  
id = true;  // error
```

7. Generics

A way to write reusable code that works with any type. `<T>` is a placeholder.

```
function wrapInArray<T>(value: T): T[] {  
  return [value];  
}
```

You'll see this with Angular's `Observable<User>` , `HttpClient.get<User[]>()` , etc.

8. Decorators

A function prefixed with `@` that *modifies* a class or member.

```
@Component({  
  selector: 'app-hello',  
  template: '<h1>Hello</h1>'  
})  
export class HelloComponent {}
```

You'll see `@Component` , `@Injectable` , `@Input` , `@Output` — same idea.

Topic 2 — Angular CLI & Project Structure

The **Angular CLI** is a command-line tool that creates, builds, and manages Angular projects.

1. Installation

```
npm install -g @angular/cli
```

Verify: `ng version` should show v20 or v21.

2. Creating a New Project

```
ng new my-app
```

Prompts:

- **Stylesheet?** `SCSS`
- **SSR?** `No (for now)`
- **Zoneless?** `No (for now)`

Then:

```
cd my-app  
ng serve
```

Open `http://localhost:4200` .

3. Folder Structure

```
my-app/  
  node_modules/  
  src/  
    app/  
      app.ts          root component class  
      app.html        root component template  
      app.scss        root component styles  
      app.config.ts   app-wide config  
      app.routes.ts   routing  
    index.html  
    main.ts          entry point  
    styles.scss      global styles  
  angular.json  
  package.json  
  tsconfig.json
```

The 4 files you'll touch 99% of the time: `app.ts` , `app.html` , `app.routes.ts` , `app.config.ts` .

4. How a Page Loads

1. Browser loads `index.html` (has `<app-root></app-root>`).
2. `main.ts` runs `bootstrapApplication(App, appConfig)` .
3. Angular finds `<app-root>` and renders the `App` component there.
4. Component tree branches out from there.

5. Generating Files

```
ng g c header      # component  
ng g s services/user  # service  
ng g d directives/highlight # directive  
ng g p pipes/currency  # pipe  
ng g g guards/auth    # guard
```

6. Common CLI Commands

Command	What it does
<code>ng serve</code>	Dev server with hot reload
<code>ng serve -o</code>	Same, auto-opens browser
<code>ng build</code>	Production build <code>/dist</code>
<code>ng generate <type> <name></code>	Create files
<code>ng test</code>	Run tests

7. The Component File

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.scss']
})
export class App {
  title = 'my-app';
}
```

- `@Component({...})` — decorator that says “this is a component”
- `selector` — the HTML tag (`<app-root>`)
- `imports` — other components/directives this uses
- `export class App {}` — the class with logic

Topic 3 — Components

Components are the **building blocks** of every Angular app. A component is **3 things bundled**:

1. **A class** — the logic (TS)
2. **A template** — the HTML

3. **Styles** — the CSS (scoped to this component)

Plus a `@Component` decorator and a **selector** (its HTML tag name).

1. Anatomy

`ng g c product-card` generates:

```
// product-card.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-product-card',
  templateUrl: './product-card.html',
  styleUrls: ['./product-card.scss']
})
export class ProductCard {}
```

To use elsewhere: `<app-product-card></app-product-card>` .

2. A Real Component

```
@Component({
  selector: 'app-product-card',
  templateUrl: './product-card.html',
  styleUrls: ['./product-card.scss']
})
export class ProductCard {
  productName = 'Wireless Headphones';
  price = 2999;
  inStock = true;

  buy(): void {
    console.log(`Buying ${this.productName}`);
  }
}
```

```
<div class="card">
  <h2>{{ productName }}</h2>
  <p>Price: {{ price }}</p>
  <p>{{ inStock ? 'Available' : 'Sold out' }}</p>
  <button (click)="buy()">Buy Now</button>
</div>
```

- {{ productName }} — **interpolation**
- (click)="buy()" — **event binding**

3. Using a Component Inside Another

The parent's TS file must import it:

```
import { ProductCard } from './product-card/product-card';

@Component({
  selector: 'app-root',
  imports: [ProductCard], // import here
  templateUrl: './app.html'
})
export class App {}
```

Then in app.html :

```
<h1>My Store</h1>
<app-product-card></app-product-card>
```

The imports: [...] array is a feature of **standalone components** (the modern way). No NgModule needed.

4. The @Component Decorator Options

```
@Component({
  selector: 'app-product-card', // HTML tag
  templateUrl: './product-card.html', // OR template: '<h1>...</h1>'
  styleUrls: ['./product-card.scss'], // OR styles: ['h1 { color: red; }']
})
```

```
imports: [SomeOtherComponent]
})
```

Naming convention: Always prefix selectors (`app-`).

5. Properties & Methods

```
export class ProductCard {
  count = 0;

  increment(): void { this.count++; }
  reset(): void { this.count = 0; }
}
```

In the template:

```
<p>Count: {{ count }}</p>
<button (click)="increment()">+</button>
```

Anything the template uses must be on the class as a property or method.

6. Style Scoping

Styles in `product-card.scss` only apply to *that* component. Angular adds a unique attribute to the HTML and rewrites your CSS. You don't think about it — your CSS won't leak.

Global styles go in `src/styles.scss` .

7. Inline vs External Templates

External (default, for anything more than 2 lines):

```
templateUrl: './product-card.html',
styleUrl: './product-card.scss'
```

Inline (only for tiny components):

```
template: `<span class="badge">{{ label }}</span>`,
styles: [`.badge { background: yellow; padding: 4px; }`]
```

8. The Component Tree

```
App (root)
  Header
    Logo
  ProductList
    ProductCard
    ProductCard
  Footer
```

Each component does one thing. If it's getting big (>200 lines, 3+ concerns), split it.

Topic 4 — Templates & Data Binding

Four types of data binding. **Memorize the four** — this is *the* most-asked junior interview question.

#	Type	Direction	Syntax	Example
1	Interpolation	Class Template	<code>{{ }}</code>	<code>{{ name }}</code>
2	Property binding	Class Template	<code>[]</code>	<code>[disabled]="isLoading"</code>
3	Event binding	Template Class	<code>()</code>	<code>(click)="save()"</code>
4	Two-way binding	Both	<code>[()]</code>	<code>[(ngModel)]="email"</code>

Mnemonic: `{{ }}` `[]` `()` `[( )]` — banana in a box.

1. Interpolation `{{ }}`

```
export class App {
  username = 'Ravi';
  age = 25;
```

```
user = { name: 'Asha', city: 'Chennai' };  
}
```

```
<p>Hello, {{ username }}</p>      <!-- Hello, Ravi -->  
<p>Age: {{ age }}</p>             <!-- Age: 25 -->  
<p>City: {{ user.city }}</p>      <!-- City: Chennai -->  
<p>Next year: {{ age + 1 }}</p>    <!-- Next year: 26 -->  
<p>Upper: {{ username.toUpperCase() }}</p>
```

Inside `{{ }}` : property access, method calls, arithmetic, ternary. NOT: assignments, `new` , `++` / `--` , `if` / `for` .

Interpolation **always produces a string**.

2. Property Binding []

```
export class App {  
  imgUrl = 'https://example.com/cat.jpg';  
  isDisabled = true;  
}
```

```
<img [src]="imgUrl" />  
<button [disabled]="isDisabled">Submit</button>
```

Why not `disabled="isDisabled"` ? That sets the literal string `"isDisabled"` (always truthy always disabled). `[disabled]` evaluates as TS, so `isDisabled` stays a boolean.

Quick rule:

- `src="cat.jpg"` static
- `[src]="imgUrl"` dynamic from class
- `src="{{ imgUrl }}"` also works, but `[src]` is preferred

Common uses:

```
<button [disabled]="!form.valid">Submit</button>  
<div [class.active]="isActive">...</div>  
<div [style.color]="textColor">...</div>
```

3. Event Binding ()

```
export class App {  
  count = 0;  
  increment(): void { this.count++; }  
  onKey(event: KeyboardEvent): void { console.log(event.key); }  
}
```

```
<button (click)="increment()">Clicked {{ count }} times</button>  
<input (keyup)="onKey($event)" />  
<form (submit)="save()">...</form>
```

`$event` holds the DOM event object — automatically available inside event bindings.

Common events: (click) , (input) , (change) , (focus) , (blur) , (keyup) , (submit) , (mouseenter) .

4. Two-Way Binding [()]

For form inputs where a variable should always match what the user typed.

```
import { FormsModule } from '@angular/forms';  
  
@Component({  
  imports: [FormsModule], // required for ngModel  
  templateUrl: './app.html'  
})  
export class App {  
  email = '';  
}
```

```
<input [(ngModel)]="email" />  
<p>You typed: {{ email }}</p>
```

`[(ngModel)]` is shorthand for:

```
<input [value]="email" (input)="email = $any($event.target).value" />
```

Requires **FormsModule** in component imports. Otherwise: *"Can't bind to 'ngModel'".*

5. Class & Style Bindings

```
<!-- Toggle a single class -->
<div [class.active]="isActive">...</div>

<!-- From an object -->
<div [class]="{ active: isActive, disabled: isDisabled }">...</div>

<!-- Single inline style -->
<div [style.color]="textColor">...</div>
<div [style.font-size.px]="fontSize">...</div>

<!-- Multiple styles -->
<div [style]="{ color: textColor, fontSize: fontSize + 'px' }">...</div>
```

6. Putting It All Together

```
@Component({
  selector: 'app-greeter',
  imports: [FormsModule],
  template: `
    <h2>Hello, {{ name || 'stranger' }}!</h2>
    <input [(ngModel)]="name" placeholder="Your name" />
    <button (click)="reset()" [disabled]="!name">Reset</button>
    <p [class.warning]="name.length > 20">Length: {{ name.length }}</p>
  `,
  styles: [`.warning { color: red; }`]
})
export class Greeter {
  name = "";
  reset(): void { this.name = ""; }
}
```

7. Template Reference Variables

```
<input #emailInput type="email" />
<button (click)="logValue(emailInput.value)">Log</button>
```

`#emailInput` is a local variable pointing to the DOM element.

Common Mistakes

1. **Forgetting parentheses on event handlers.** `(click)="save"` does nothing. `(click)="save()"` is correct.
 2. **Using `[]` for static values.** `[type]="text"` works but pointless. `type="text"` is cleaner.
 3. **Two-way binding without `FormsModule`**. Error: *"Can't bind to 'ngModel'"*.
-

Topic 5 — Control Flow (`@if` , `@for` , `@switch`)

Modern Angular has three built-in control flow blocks: `@if` , `@for` , `@switch` . Before Angular 17, you used `*ngIf` , `*ngFor` , `*ngSwitch` (still work in older code).

Use `@if` / `@for` / `@switch` for new code. Recognize the old syntax in interviews.

1. `@if` — Conditional Rendering

```
isLoggedIn = true;
user = { name: 'Ravi', isAdmin: false };
```

```
@if (isLoggedIn) {
  <p>Welcome back, {{ user.name }}!</p>
}
```

If false, the element doesn't exist in the DOM (not just hidden).

`@else` and `@else if`


```
@if (user.isAdmin) {
  <p>Admin panel</p>
} @else if (isLoggedIn) {
  <p>User dashboard</p>
} @else {
  <p>Please log in</p>
}
```

Storing the result (as)

```
@if (getUser(); as user) {
  <p>Hello, {{ user.name }}</p>
}
```

2. @for — Loops

```
products = [
  { id: 1, name: 'Pen', price: 20 },
  { id: 2, name: 'Notebook', price: 80 }
];
```

```
@for (product of products; track product.id) {
  <div class="card">
    <h3>{{ product.name }}</h3>
    <p> {{ product.price }}</p>
  </div>
}
```

track is MANDATORY

Unlike *ngFor , @for requires track . Tells Angular how to identify items.

- track product.id — best
- track \$index — fallback when no unique ID
- track product — works but inefficient

Loop variables

```
@for (item of items; track item.id; let i = $index, isFirst = $first) {  
  <p>{{ i }}: {{ item.name }} {{ isFirst ? '(first)' : '' }}</p>  
}
```

Variable	Meaning
<code>\$index</code>	0, 1, 2...
<code>\$first</code>	true if first
<code>\$last</code>	true if last
<code>\$even</code> / <code>\$odd</code>	even/odd
<code>\$count</code>	total

@empty block

```
@for (todo of todos; track todo.id) {  
  <li>{{ todo.title }}</li>  
} @empty {  
  <li>No todos yet — add one!</li>  
}
```

3. @switch

```
status: 'loading' | 'success' | 'error' = 'loading';
```

```
@switch (status) {  
  @case ('loading') { <p>Loading...</p> }  
  @case ('success') { <p>Data loaded!</p> }  
  @case ('error') { <p>Something went wrong.</p> }  
  @default { <p>Unknown state</p> }  
}
```

No fall-through, no `break` needed.

4. Nesting

```
@if (users.length > 0) {  
  <ul>  
    @for (user of users; track user.id) {  
      <li>  
        {{ user.name }}  
        @if (user.isAdmin) {  
          <span class="badge">Admin</span>  
        }  
      </li>  
    }  
  </ul>  
} @else {  
  <p>No users found.</p>  
}
```

5. The Old Syntax (Recognize, Don't Write)

```
<p *ngIf="isLoggedIn">Welcome!</p>  
  
<div *ngFor="let product of products; trackBy: trackById">  
  {{ product.name }}  
</div>  
  
<div [ngSwitch]="status">  
  <p *ngSwitchCase="'loading'">Loading...</p>  
  <p *ngSwitchDefault>Unknown</p>  
</div>
```

Why new is better:

1. Built into template compiler — no `CommonModule` import.
2. Better performance (up to 90% faster).
3. Better type narrowing.
4. `@empty` block (no equivalent in `*ngFor`).
5. `track` is mandatory.

Interview Soundbite

"The new `@if`, `@for`, `@switch` syntax replaces `*ngIf` / `*ngFor` / `*ngSwitch`. It's built into the template engine — better performance, no `CommonModule` needed, and `track` in `@for` is mandatory, preventing performance bugs."

Common Mistakes

1. **Forgetting `track` in `@for`** — won't compile.
 2. **`@if` removes element from DOM** — for visual hide, use `[hidden]` or `[style.display]`.
-

Topic 6 — Directives

A **directive** is a class that adds behavior to an existing DOM element. Components are technically a special kind of directive (they have a template). The other two: **attribute directives** and **structural directives**.

The Three Types

Type	What	Example
Component	Directive with template	<code><app-card></code>
Attribute directive	Changes appearance/behavior	<code>[ngClass]</code> , custom
Structural directive	Adds/removes elements	<code>*ngIf</code> , <code>*ngFor</code> (legacy)

1. Built-in Attribute Directives

`ngClass` — toggle multiple classes

```
<!-- Object syntax -->
<div [ngClass]="{ active: isActive, disabled: isDisabled }">...</div>
```

```
<!-- Array syntax -->
<div [ngClass]="['card', size]">...</div>
```

Modern alternative — `[class]` binding:

```
<div [class.active]="isActive" [class.disabled]="isDisabled">...</div>
<div [class]="{ active: isActive, disabled: isDisabled }">...</div>
```

`[class]` now covers most `ngClass` use cases.

ngStyle

```
<div [ngStyle]="{ color: textColor, 'font-size.px': fontSize }">...</div>
```

Modern: `[style]` :

```
<div [style.color]="textColor" [style.font-size.px]="fontSize">...</div>
```

Rule: Use `[class.x]` and `[style.x]` for simple cases. `ngClass` / `ngStyle` only for whole-object application.

2. The Old Structural Directives

```
<p *ngIf="isLoggedIn">Welcome</p>

<li *ngFor="let item of items; trackBy: trackById; let i = index">
  {{ i }}: {{ item.name }}
</li>

<div [ngSwitch]="status">
  <p *ngSwitchCase="'loading'">Loading</p>
</div>
```

The asterisk is shorthand for wrapping in `<ng-template>` . `*ngIf="cond"` is shorthand for `<ng-template [ngIf]="cond"><p>...</p></ng-template>` .

3. Custom Attribute Directives

ng g d highlight

```
import { Directive, ElementRef, inject, Input } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
  host: {
    '(mouseenter)': 'onEnter()',
    '(mouseleave)': 'onLeave()'
  }
})
export class Highlight {
  @Input() appHighlight = 'yellow';
  private el = inject(ElementRef);

  onEnter() { this.el.nativeElement.style.backgroundColor = this.appHighlight; }
  onLeave() { this.el.nativeElement.style.backgroundColor = ""; }
}
```

Use it:

```
<p appHighlight>Default yellow</p>
<p [appHighlight]="lightblue">Light blue</p>
```

Breakdown:

- selector: '[appHighlight]' — `[]` means “match an attribute”
- ElementRef — gives DOM element access via `el.nativeElement`
- host — listen for events on the host element
- @Input() — accept a value from the parent

4. When to Use a Directive vs Component

Component when...	Directive when...
Need a template	Add behavior to existing elements
Self-contained UI	Reusable across element types

card, modal, navbar

tooltip, autofocus, click-outside

5. A Useful Directive: Auto-Focus

```
@Directive({ selector: '[appAutofocus]' })
export class Autofocus implements AfterViewInit {
  private el = inject(ElementRef);

  ngAfterViewInit(): void {
    this.el.nativeElement.focus();
  }
}
```

```
<input appAutofocus placeholder="Auto-focused on load" />
```

Common Mistakes

1. **Wrong selector syntax.** `selector: 'appHighlight'` matches `<appHighlight>`. You want `selector: '[appHighlight]'` (brackets = attribute).
2. **Forgetting to import** — directives, like components, must be in `imports: [...]`.

Interview Soundbite

“Three kinds of directives. Components are directives with a template. Attribute directives change appearance or behavior — `ngClass`, custom directives like `appHighlight`. Structural directives add or remove elements — `*ngIf`, `*ngFor`, `*ngSwitch` in older code, or `@if`, `@for`, `@switch` as the modern equivalents.”

Topic 7 — Pipes

A **pipe** is a function for transforming a value for display. Syntax uses `|` :

```
{{ value | pipeName }}
```

1. The Basics

```
<p>{{ price | currency }}</p>      <!-- $1,234.50 -->
<p>{{ today | date }}</p>          <!-- May 7, 2026 -->
<p>{{ name | uppercase }}</p>      <!-- RAVI KUMAR -->
<p>{{ name | titlecase }}</p>      <!-- Ravi Kumar -->
```

2. Built-in Pipes

Text

```
{{ 'hello' | uppercase }}      <!-- HELLO -->
{{ 'HELLO' | lowercase }}      <!-- hello -->
{{ 'hello world' | titlecase }} <!-- Hello World -->
{{ 'long text' | slice:0:8 }}   <!-- long tex -->
```

Number

```
{{ 3.14159 | number }}          <!-- 3.142 -->
{{ 3.14159 | number:'1.2-2' }}  <!-- 3.14 -->
{{ 0.25 | percent }}            <!-- 25% -->
{{ 1234.5 | currency }}          <!-- $1,234.50 -->
{{ 1234.5 | currency:'INR':'symbol' }} <!-- 1,234.50 -->
```

Date

```
{{ today | date }}              <!-- May 7, 2026 -->
{{ today | date:'short' }}       <!-- 5/7/26, 3:45 PM -->
{{ today | date:'dd/MM/yyyy' }}  <!-- 07/05/2026 -->
{{ today | date:'HH:mm:ss' }}    <!-- 15:45:30 -->
```

json (debugging gold)

```
<pre>{{ user | json }}</pre>
```


keyvalue

```
config = { theme: 'dark', lang: 'en' };
```

```
@for (item of config | keyvalue; track item.key) {  
  <p>{{ item.key }}: {{ item.value }}</p>  
}
```

async pipe — covered fully in topic 17

3. Arguments

```
{{ price | currency:'EUR' }}          <!-- one arg -->  
{{ price | currency:'EUR':'symbol' }} <!-- two args -->  
{{ price | currency:'EUR':'symbol':'1.2-2' }} <!-- three args -->
```

4. Chaining

```
{{ name | lowercase | titlecase }}  
{{ today | date:'fullDate' | uppercase }}
```

5. The async Pipe — The Most Important One

```
users$ = inject(HttpClient).get<User[]>('/api/users');
```

```
@for (user of users$ | async; track user.id) {  
  <p>{{ user.name }}</p>  
}
```

What async does:

1. Subscribes to the Observable.
2. Returns latest value to template.
3. Auto-unsubscribes on destroy.

Without it, you'd manage `Subscription` and `ngOnDestroy` manually. **Always prefer `async` pipe in templates.**

6. Custom Pipes

```
ng g p pipes/truncate
```

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'truncate' })
export class TruncatePipe implements PipeTransform {
  transform(value: string, limit: number = 50, ellipsis: string = '...'): string {
    if (!value) return '';
    return value.length > limit ? value.slice(0, limit) + ellipsis : value;
  }
}
```

```
@Component({
  imports: [TruncatePipe],
  template: `<p>{{ longText | truncate:20 }}</p>`
})
```

7. Pure vs Impure Pipes

Pure (default): Only re-runs when input *reference* changes. Fast.

```
items = [1, 2, 3];
items.push(4); // pure pipe won't re-run
items = [...items, 4]; // pure pipe will re-run (new reference)
```

Impure: Re-runs on every CD cycle.

```
@Pipe({ name: 'myPipe', pure: false })
```

`async` pipe is impure (must be — `async`). Most others are pure.

8. Pipes vs Methods

Don't:

```
<p>{{ formatPrice(price) }}</p>
```

Methods run on every CD cycle (potentially hundreds of times).

Always prefer a pipe.

Common Mistakes

1. **Forgetting to import pipes** — `CurrencyPipe` , `DatePipe` , `AsyncPipe` must be in `imports: [...]` .
2. **Forgetting async pipe** — leads to manual subscriptions and leaks.
3. **Mutating arrays** with pure pipes — replace, don't mutate.

Interview Soundbites

Q: What's a pipe?

"A function that transforms a value for display, used with the `|` syntax. Built-in ones include `currency` , `date` , `async` , `uppercase` . Custom pipes implement `PipeTransform` ."

Q: Why is `async` important?

"It auto-subscribes and auto-unsubscribes Observables in templates, preventing memory leaks. The idiomatic way to consume Observables in Angular."

Topic 8 — Component Communication (`@Input` , `@Output`)

THE most-asked junior interview topic. The short answer:

- **Parent Child:** `@Input()`
- **Child Parent:** `@Output()` + `EventEmitter`
- **Unrelated components:** Shared service (covered in topic 10)

1. @Input() — Parent Child

Child:

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-product-card',
  template: `
    <div class="card">
      <h3>{{ product.name }}</h3>
      <p> {{ product.price }}</p>
    </div>
  `,
})
export class ProductCard {
  @Input() product!: { name: string; price: number };
}
```

Parent:

```
@Component({
  imports: [ProductCard],
  template: `
    @for (p of products; track p.name) {
      <app-product-card [product]="p"></app-product-card>
    }
  `,
})
export class ProductList {
  products = [
    { name: 'Pen', price: 20 },
    { name: 'Notebook', price: 80 }
  ];
}
```

Critical syntax: `[product]="p"` — square brackets, property binding.

Renaming

```
@Input('item') product!: Product;
```

```
<app-product-card [item]="p"></app-product-card>
```

Required inputs (Angular 16+)

```
@Input({ required: true }) product!: Product;
```

The !

Tells TypeScript: *"I promise this will be assigned."* Without it, TS complains.

2. @Output() — Child Parent

Child:

```
import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-product-card',
  template: `
    <button (click)="onBuy()">Buy</button>
  `
})
export class ProductCard {
  @Input() product!: { name: string; price: number };
  @Output() buyClicked = new EventEmitter<{ name: string; price: number }>();

  onBuy(): void {
    this.buyClicked.emit(this.product);
  }
}
```

Parent:

```
@Component({
  imports: [ProductCard],
  template: `
    @for (p of products; track p.name) {
      <app-product-card
        [product]="p"
        (buyClicked)="handleBuy($event)">
      </app-product-card>
    }
  `,
})
export class ProductList {
  products = [ /* ... */ ];

  handleBuy(item: { name: string; price: number }): void {
    console.log('Bought', item.name);
  }
}
```

Critical syntax: `(buyClicked)="handleBuy($event)"` — parens, event binding. `$event` is whatever was passed to `.emit(...)`.

3. Complete Example

```
// product-card.ts
@Component({
  selector: 'app-product-card',
  template: `
    <div class="card">
      <h3>{{ product.name }}</h3>
      <p> {{ product.price }}</p>
      <button (click)="onBuy()">Buy</button>
      <button (click)="onRemove()">Remove</button>
    </div>
  `,
})
export class ProductCard {
```

```

@Input({ required: true }) product!: Product;
@Output() buy = new EventEmitter<Product>();
@Output() remove = new EventEmitter<number>();

onBuy() { this.buy.emit(this.product); }
onRemove() { this.remove.emit(this.product.id); }
}

```

```

// product-list.ts
@Component({
  imports: [ProductCard],
  template: `
    <h2>Cart Total: {{ total }}</h2>
    @for (p of products; track p.id) {
      <app-product-card
        [product]="p"
        (buy)="onBuy($event)"
        (remove)="onRemove($event)">
      </app-product-card>
    }
  `,
})
export class ProductList {
  products: Product[] = [ /* ... */ ];
  total = 0;

  onBuy(p: Product) { this.total += p.price; }
  onRemove(id: number) { this.products = this.products.filter(p => p.id !== id); }
}

```

4. Modern: Signal Inputs/Outputs (Angular 17.1+)

```

import { Component, input, output } from '@angular/core';

@Component({
  selector: 'app-product-card',
  template: `
    <h3>{{ product().name }}</h3>

```

```

    <button (click)="buy.emit(product())">Buy</button>
    ,
  })
  export class ProductCard {
    product = input.required<Product>(); // signal input
    buy = output<Product>();           // signal output
  }

```

Differences:

- Functions, not decorators
- Read via `product()` (it's a signal)
- No `EventEmitter` import

Decorator `@Input` / `@Output` still 100% valid. New code increasingly uses signal-based.

5. Unrelated Components Shared Service

```

@Injectable({ providedIn: 'root' })
export class CartService {
  itemCount = signal(0);
}

```

Both components inject the same instance. Topic 10 covers this fully.

6. Anti-Patterns

Don't mutate `@Input` in child

```

@Input() product!: Product;
increasePrice() {
  this.product.price += 10; // mutating parent's data!
}

```

Emit an event:

```

@Output() priceIncrease = new EventEmitter<number>();
increasePrice() { this.priceIncrease.emit(this.product.id); }

```


Data flows down, events flow up. Unidirectional data flow.

Don't reach into children with `ViewChild` for normal communication

`@ViewChild` is for imperative interactions, not data flow.

Cheat Sheet

Direction	Tool	Parent template syntax
Parent → Child	<code>@Input()</code>	<code>[propName]="value"</code>
Child → Parent	<code>@Output()</code> + <code>EventEmitter</code>	<code>(eventName)="handler(\$event)"</code>
Unrelated Unrelated	Shared service	(no template — both inject)

Common Mistakes

1. **Wrong bracket type** — `[]` for input, `()` for output.
2. **Forgetting `EventEmitter` initialization** — `@Output()` buy = new `EventEmitter<...>()`; .
3. **Not importing the child** — must be in parent's `imports: [...]` .
4. **Mutating `@Input` objects** — emit events instead.

Interview Soundbite

Q: How do components communicate in Angular?

"Three ways. Parent-to-child via `@Input()` , where the parent passes data with `[input]="value"` . Child-to-parent via `@Output()` and `EventEmitter` — child emits, parent listens with `(event)="handler($event)"` . For unrelated components, a shared service with `providedIn: 'root'` that both inject."

Topic 9 — Lifecycle Hooks

A component goes through a lifecycle: created, inputs set, rendered, possibly updated, destroyed. **Lifecycle hooks** let you run code at each moment.

The two you'll use 95% of the time: **ngOnInit** and **ngOnDestroy** .

1. The Full List (in order)

#	Hook	When
1	constructor	Class instantiated
2	ngOnChanges	Before ngOnInit , then on every @Input change
3	ngOnInit	Once, after first ngOnChanges
4	ngDoCheck	Every CD cycle (rare)
5	ngAfterContentInit	After projected content is set
6	ngAfterContentChecked	Every cycle, after content checked
7	ngAfterViewInit	After view & child views are ready
8	ngAfterViewChecked	Every cycle, after view checked
9	ngOnDestroy	Just before destruction

Memorize the bolded two. Know **ngOnChanges** and **ngAfterViewInit** exist.

2. The Pattern

```
import { Component, OnInit, OnDestroy } from '@angular/core';

@Component({ /* ... */ })
export class MyComponent implements OnInit, OnDestroy {
  ngOnInit(): void { /* runs once on init */ }
  ngOnDestroy(): void { /* runs once on destroy */ }
}
```

`implements OnInit` is optional but recommended for type safety.

3. **ngOnInit** — The Workhorse

```
export class UserList implements OnInit {
  private http = inject(HttpClient);
  users: User[] = [];

  ngOnInit(): void {
    this.http.get<User[]>('/api/users').subscribe(data => {
      this.users = data;
    });
  }
}
```

What goes here:

- Initial data fetching
- Setting up form controls
- Subscribing to observables
- Reading `@Input` for setup

What does NOT belong:

- DOM manipulation (use `ngAfterViewInit`)
- Reading `@ViewChild` references (use `ngAfterViewInit`)

4. Why `ngOnInit` and Not the Constructor?

Guaranteed interview question.

```
constructor() {
  console.log(this.product); // undefined! @Inputs aren't set yet
}

ngOnInit() {
  console.log(this.product); // present, ready
}
```

- **Constructor** runs when class is *instantiated* — Inputs not yet set.
- **`ngOnInit`** runs *after* Inputs are set.

Interview answer: “The constructor is for dependency injection setup. `ngOnInit` runs after Angular has initialized inputs, so it’s the right place for logic that depends on them.”

5. ngOnDestroy — Cleanup

```
export class LiveFeed implements OnInit, OnDestroy {
  private timerId?: number;
  private subscription?: Subscription;

  ngOnInit(): void {
    this.timerId = window.setInterval(() => console.log('tick'), 1000);
    this.subscription = this.dataService.feed$.subscribe(/* ... */);
  }

  ngOnDestroy(): void {
    clearInterval(this.timerId);
    this.subscription?.unsubscribe();
  }
}
```

What needs cleanup:

- Manual Observable subscriptions (or use async pipe)
- setInterval / setTimeout
- addEventListener on window / document
- WebSockets

Skip cleanup memory leak.

6. Modern: DestroyRef and takeUntilDestroyed()

```
import { DestroyRef, inject } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

export class LiveFeed {
  private destroyRef = inject(DestroyRef);

  constructor() {
    this.destroyRef.onDestroy(() => clearInterval(this.timerId));

    this.dataService.feed$
      .pipe(takeUntilDestroyed())
```

```
.subscribe(/* ... */);  
}  
}
```

The modern way. `ngOnDestroy` still valid.

7. `ngOnChanges` — Reacting to `@Input` Changes

```
import { OnChanges, SimpleChanges } from '@angular/core';  
  
export class UserCard implements OnChanges {  
  @Input() userId!: number;  
  
  ngOnChanges(changes: SimpleChanges): void {  
    if (changes['userId']) {  
      console.log('previous:', changes['userId'].previousValue);  
      console.log('current:', changes['userId'].currentValue);  
      this.fetchUser(this.userId);  
    }  
  }  
}
```

Important: Only fires on **reference** change. Mutating `this.user.name` doesn't trigger it.

8. `ngAfterViewInit` — DOM Access

```
export class Chart implements AfterViewInit {  
  @ViewChild('canvas') canvasRef!: ElementRef<HTMLCanvasElement>;  
  
  ngAfterViewInit(): void {  
    const ctx = this.canvasRef.nativeElement.getContext('2d');  
    // initialize chart library  
  }  
}
```

`@ViewChild` is undefined in `ngOnInit` — the view doesn't exist yet.

9. Order Recap

When component is created:

```
constructor  ngOnChanges  ngOnInit  ngDoCheck  ngAfterContentInit  
            ngAfterContentChecked  ngAfterViewInit  ngAfterViewChecked
```

When `@Input` changes:

```
ngOnChanges  ngDoCheck  ngAfterContentChecked  ngAfterViewChecked
```

When destroyed:

```
ngOnDestroy
```

Common Mistakes

1. **Initialization in constructor instead of `ngOnInit`** . Inputs aren't ready.
2. **Forgetting to clean up subscriptions**. Memory leaks.
3. **Trying to read `@ViewChild` in `ngOnInit`** . Undefined. Use `ngAfterViewInit` .
4. **Heavy work in `ngDoCheck` or "Checked" hooks**. Runs on every CD cycle.
5. **Mutating an input expecting `ngOnChanges` to fire**. Only fires on reference change.

Interview Soundbites

Q: Constructor vs `ngOnInit` ?

"Constructor runs on instantiation — used for DI. `ngOnInit` runs after Angular sets `@Input` properties — the right place for initialization logic."

Q: When would you use `ngOnDestroy` ?

"For cleanup. Unsubscribing Observables, clearing intervals/timeouts, removing event listeners. Modern alternative: `takeUntilDestroyed()` ."

Q: When would you use `ngAfterViewInit` ?

"When you need DOM access or `@ViewChild` references. The view doesn't exist in `ngOnInit` but does in `ngAfterViewInit` ."

Topic 10 — Services

A **service** is a TypeScript class that holds reusable logic — data fetching, shared state, business logic.

Golden rule: Components handle UI; services handle everything else.

1. Why Services Exist

Without services, every component fetches its own data. Bug fixes need updating 5 places. With services:

```
@Injectable({ providedIn: 'root' })
export class UserService {
  private http = inject(HttpClient);
  getUsers() { return this.http.get<User[]>('/api/users'); }
}

// In any component
private userService = inject(UserService);
this.userService.getUsers().subscribe(...)
```

Logic in one place. **Separation of concerns.**

2. Creating a Service

```
ng g s services/user
```

```
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class UserService {
  constructor() {}
}
```

3. `providedIn` Options

Option	Meaning
'root'	One singleton, app-wide (99% of cases)
'platform'	Across multiple Angular apps on the page (rare)
'any'	New instance per lazy-loaded module/route

providedIn: 'root' means:

- **One instance** shared by all consumers
- **Tree-shakeable** — dropped from build if unused

4. Using a Service — Two Ways

Modern: `inject()`

```
import { Component, inject } from '@angular/core';

@Component({ /* ... */ })
export class UserList {
  private userService = inject(UserService);
}
```

Classic: constructor injection

```
@Component({ /* ... */ })
export class UserList {
  constructor(private userService: UserService) {}
}
```

Both work. `inject()` is newer (Angular 14+), preferred in new code, required in some places (functional guards, interceptors).

5. Three Common Patterns

A. Data fetching


```

@Injectable({ providedIn: 'root' })
export class ProductService {
  private http = inject(HttpClient);
  private apiUrl = '/api/products';

  getAll() { return this.http.get<Product[]>(this.apiUrl); }
  getById(id: number) { return this.http.get<Product>(`${this.apiUrl}/${id}`); }
  create(p: Omit<Product, 'id'>) { return this.http.post<Product>(this.apiUrl, p); }
  update(id: number, p: Partial<Product>) { return this.http.patch<Product>(`${this.apiUrl}/${id}`, p); }
  delete(id: number) { return this.http.delete(`${this.apiUrl}/${id}`); }
}

```

One service per resource. Most common pattern.

B. Shared state

```

@Injectable({ providedIn: 'root' })
export class CartService {
  itemCount = signal(0);
  items = signal<CartItem[]>([]);

  addItem(item: CartItem) {
    this.items.update(list => [...list, item]);
    this.itemCount.set(this.items().length);
  }
}

```

Both `Navbar` and `AddToCartButton` inject the same instance same state.

(Legacy: `BehaviorSubject` instead of `signal` . Same idea.)

C. Plain logic

```

@Injectable({ providedIn: 'root' })
export class FormatService {
  formatCurrency(amount: number, currency = 'INR'): string {
    return new Intl.NumberFormat('en-IN', { style: 'currency', currency }).format(amount);
  }
}

```

```
}  
}
```

6. Services Injecting Services

```
@Injectable({ providedIn: 'root' })  
export class OrderService {  
  private http = inject(HttpClient);  
  private cart = inject(CartService);  
  private auth = inject(AuthService);  
  
  checkout() {  
    const items = this.cart.items();  
    const userId = this.auth.currentUser()?.id;  
    return this.http.post('/api/orders', { items, userId });  
  }  
}
```

Composition is encouraged. Each service does one thing well.

7. Realistic Example

```
// services/cart.ts  
@Injectable({ providedIn: 'root' })  
export class CartService {  
  private _items = signal<CartItem[]>([]);  
  
  items = this._items.asReadonly();  
  count = computed(() => this._items().reduce((s, i) => s + i.quantity, 0));  
  total = computed(() => this._items().reduce((s, i) => s + i.price * i.quantity, 0));  
  
  add(item: CartItem) {  
    this._items.update(list => {  
      const existing = list.find(i => i.id === item.id);  
      if (existing) {  
        return list.map(i => i.id === item.id ? { ...i, quantity: i.quantity + 1 } : i);  
      }  
      return [...list, { ...item, quantity: 1 }];  
    });  
  }  
}
```

```

    });
  }

  remove(id: number) { this._items.update(list => list.filter(i => i.id !== id)); }
  clear() { this._items.set([]); }
}

```

```

// navbar.ts
@Component({
  selector: 'app-navbar',
  template: `
    <nav>
      <span> Cart ({{ cart.count() }}) — {{ cart.total() }}</span>
    </nav>
  `,
})
export class Navbar {
  cart = inject(CartService);
}

```

Navbar and ProductCard are unrelated in the component tree, but both see the same cart state because of `providedIn: 'root'`.

8. Public API, Private Internals

```

@Injectable({ providedIn: 'root' })
export class CartService {
  // Private — implementation
  private _items = signal<CartItem[]>([]);
  private http = inject(HttpClient);

  // Public — what consumers use
  items = this._items.asReadonly();
  add(item: CartItem) { /* ... */ }
  remove(id: number) { /* ... */ }
}

```

Components go through `add` / `remove` / `clear` , not directly to `_items` . Validation lives in one place; underlying storage can change.

9. When NOT to Use a Service

- UI state for a single component (open/closed, current input value) component
- DOM manipulation component
- Wrapper around one trivial function just a function

Rule: Service justifies its existence if it's shared, async, or stateful.

Common Mistakes

1. **Forgetting `providedIn: 'root'`** *"NullInjectorError: No provider".*
2. **Expecting separate instances** — `providedIn: 'root'` is one shared instance.
3. **`new UserService()` in a component** — defeats DI.
4. **Subscribing in services** — services return Observables; components subscribe.
5. **One giant `MainService`** — split by responsibility.

Interview Soundbites

Q: What is a service?

"A class with `@Injectable` that holds reusable logic — data fetching, shared state, business logic. Components inject it. Keeps components focused on UI, centralizes logic that would otherwise duplicate."

Q: What does `providedIn: 'root'` mean?

"One singleton instance app-wide. Every consumer gets the same one. Also tree-shakeable — dropped if unused."

Q: How do unrelated components communicate?

"Shared service — both inject the same singleton. With signals or `BehaviorSubject` , the service holds reactive state both components observe."

Topic 11 — Dependency Injection (DI)

DI is one of Angular's deepest topics. As a junior: understand the basics, the vocabulary, common patterns.

1. What DI Is

A design pattern where a class **doesn't create what it needs** — those things are *given* to it.

Without DI:

```
class UserList {  
  private http: HttpClient;  
  constructor() {  
    this.http = new HttpClient(/* complex setup */);  
  }  
}
```

With DI:

```
class UserList {  
  private http = inject(HttpClient);  
  // Angular gives me HttpClient. I don't care how it was made.  
}
```

The class **declares** what it needs. Angular's **injector** provides.

2. Why DI Matters

1. **Decoupling** — depends on abstractions, not implementations
2. **Testability** — swap real services with fakes in tests
3. **Singleton management** — `providedIn: 'root'` ensures one instance

3. The Three Pieces

Piece	What
Injector	Creates and tracks instances
Provider	Tells injector how to create something
Token	The “key” for lookup (usually the class)

4. Providers — Four Kinds

useClass (default)

```
@Injectable({ providedIn: 'root' })
export class UserService { /* ... */ }

// Equivalent to:
{ provide: UserService, useClass: UserService }
```

Swap for testing:

```
{ provide: UserService, useClass: MockUserService }
```

useValue — inject a fixed value

```
import { InjectionToken } from '@angular/core';

export const API_URL = new InjectionToken<string>('API_URL');

providers: [
  { provide: API_URL, useValue: 'https://api.example.com' }
]

// In a service:
private apiUrl = inject(API_URL);
```

`InjectionToken` for non-class things (config strings, feature flags). Type-safe.

useFactory — custom creation logic

```
{
  provide: Logger,
  useFactory: (config: Config) => config.production ? new ProdLogger() : new DevLogger(),
  deps: [Config]
}
```

useExisting — alias

```
{ provide: OldUserService, useExisting: UserService }
```

Both tokens point to the same instance.

5. The Injector Hierarchy

Angular has a **tree of injectors** mirroring the component tree. `inject(X)` searches:

1. Current component's injector
2. Up to parent component's injector
3. Continue up
4. Root (`providedIn: 'root'`)

First match wins.

Component-level providers

```
@Component({  
  providers: [  
    { provide: UserService, useClass: SpecialUserService }  
  ]  
})  
export class Special {  
  private userService = inject(UserService); // gets SpecialUserService  
}
```

Inside `Special` and descendants, `UserService` is `SpecialUserService`.

New instance per component

```
@Component({  
  providers: [TodoStateService]  
})
```

Every instance of this component gets its own `TodoStateService`.

Mental model:

- `providedIn: 'root'` singleton, app-wide
- Component-level `providers: [...]` one per component instance

6. Modern `inject()` vs Constructor

Both 100% valid.

```
// Constructor (classic)
constructor(private userService: UserService) {}

// inject() (modern)
private userService = inject(UserService);
```

`inject()` required in:

- Functional route guards
- Functional HTTP interceptors
- Helper functions in injection contexts

7. Modifiers

`@Optional()`

```
private logger = inject(Logger, { optional: true });
```

`null` if no provider exists, instead of erroring.

`@Self()`

Only check current injector — don't walk up.

`@SkipSelf()`

Skip current; start from parent.

Junior level: know they exist, know `@Optional` exists.

8. The `provide*()` Functions

Modern Angular APIs bundle providers:

```
providers: [  
  provideHttpClient(),  
  provideRouter(routes),  
  provideAnimations(),  
  provideClientHydration()  
]
```

Factory functions returning provider arrays. Cleaner than the old module-based setup.

9. Worked Example

```
export const API_URL = new InjectionToken<string>('API_URL');  
  
@Injectable({ providedIn: 'root' })  
export class UserService {  
  private http = inject(HttpClient);  
  private apiUrl = inject(API_URL);  
  
  getUsers() {  
    return this.http.get<User[]>(`${this.apiUrl}/users`);  
  }  
}  
  
// app.config.ts  
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideHttpClient(),  
    { provide: API_URL, useValue: 'https://api.example.com' }  
  ]  
};
```

Component asks for `UserService` injector creates it, resolves `HttpClient` and `API_URL` first.

Common Mistakes

1. **Forgetting `providedIn: 'root'`** — “`NullInjectorError`”.
2. **Expecting separate instances** when you actually want shared.
3. **`new UserService()`** — defeats DI.
4. **Circular dependencies** — extract shared logic to a third service.

Interview Soundbites

Q: What is dependency injection?

“A pattern where classes declare what they need via constructor or `inject()`, and Angular provides those dependencies. Decouples from concrete implementations, makes testable, lets Angular manage shared instances.”

Q: What does `providedIn: 'root'` do?

“Registers the service with the root injector — one singleton, app-wide. Tree-shakeable if unused.”

Q: How does Angular resolve a dependency?

“Searches injector hierarchy starting from current component, walks up to root. First match wins.”

Mental Model

Three sentences:

1. **Components and services declare what they need; the injector wires it up.**
 2. **`providedIn: 'root'` = one instance, app-wide, tree-shakeable.**
 3. **Injector searches up the tree — first match wins.**
-

Topic 12 — Standalone Components

You’ve been writing standalone components from topic 1. This topic formalizes what they are.

1. What They Are

A **standalone component** declares its own dependencies in `@Component`, no `NgModule` needed:

```
@Component({
  selector: 'app-product-list',
  imports: [ProductCard, CurrencyPipe, FormsModule],
  templateUrl: './product-list.html'
})
export class ProductList {}
```

The `imports: [...]` lists everything the template uses. That single feature replaces NgModules.

2. Three Things Can Be Standalone

Components, directives, pipes all can be standalone. Since Angular 19, **default**.

```
@Component({ /* ... */ })
@Directive({ /* ... */ })
@Pipe({ /* ... */ })
```

3. The `standalone: true` Flag

In Angular 14-18, required:

```
@Component({
  standalone: true,
  imports: [...]
})
```

In Angular 19+, default. Flag still accepted but redundant. You'll see `standalone: false` only when keeping legacy NgModule compatibility.

4. The Old World — NgModules

Before standalone:

```
@NgModule({
  declarations: [ProductList, ProductCard, TruncatePipe],
  imports: [CommonModule, FormsModule],
```

```
exports: [ProductList]
})
export class ProductModule {}
```

Components had no `imports` — the module did the wiring. Lots of boilerplate per feature.

5. Standalone vs NgModules

	Standalone	NgModule
Declares deps in...	<code>imports</code> on component	<code>declarations</code> + <code>imports</code> on module
Boilerplate	None	Module per feature
Bootstrap	<code>bootstrapApplication(App, appConfig)</code>	<code>bootstrapModule(AppModule)</code>
Lazy loading	<code>loadComponent</code>	<code>loadChildren</code> (module)

6. The Rule for `imports`

Import whatever the **template** uses:

```
imports: [
  ProductCard,    // <app-product-card>
  CurrencyPipe,   // | currency
  FormsModule,    // [(ngModel)]
  Highlight       // appHighlight directive
]
```

Forget something → clear error:

- *“'app-product-card' is not a known element”*
- *“No pipe found with name 'currency'”*
- *“Can't bind to 'ngModel'”*

You **don't** import services — they use DI, not `imports`.

7. Modern Bootstrap

```
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { App } from './app/app';
import { appConfig } from './app/app.config';

bootstrapApplication(App, appConfig);
```

```
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(),
    provideRouter(routes)
  ]
};
```

No `AppModule` . Just root component + config.

8. Lazy-Loading Standalone

Old:

```
loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule)
```

Standalone:

```
loadComponent: () => import('./admin/admin').then(m => m.Admin)
```

Single component, lazy. Topic 21 covers this fully.

9. Migrating from NgModules

You can mix:

- Standalone inside module-based templates (put it in module's `imports`)
- Module-based inside standalone (put the *module* in standalone's `imports`)

CLI migration: `ng generate @angular/core:standalone` .

10. What About CommonModule ?

Old NgModule code imported `CommonModule` everywhere — it provided `*ngIf` , `*ngFor` , `NgClass` , pipes.

In standalone code:

- `@if` / `@for` / `@switch` are built into the template engine — no import.
- `AsyncPipe` , `DatePipe` , etc. are imported individually.
- `[class]` / `[style]` bindings replace `NgClass` / `NgStyle` most of the time.

You rarely see `CommonModule` in modern code.

11. Why Angular Moved to Standalone

1. **Less boilerplate** — no “module just for one component.”
2. **Easier mental model** — deps visible on the component.
3. **Better tree-shaking.**
4. **Simpler lazy loading.**
5. **Smoother for newcomers.**
6. **Aligns with React/Vue/Solid.**

Common Mistakes

1. **Importing services into imports** — services use DI, not imports.
2. **Forgetting to import a child component** — `<app-foo>` shows as plain text.
3. **Re-exporting NgModule-style** — standalone has no transitive re-exports.
4. **Putting standalone in declarations** — only in `imports` .

Interview Soundbites

Q: What are standalone components?

“Components that declare their own dependencies in the `@Component` decorator’s `imports` array — no NgModule needed. Default in Angular 17+. Components, directives, pipes can all be standalone.”

Q: Why did Angular introduce them?

“Remove NgModule boilerplate. Components self-contain their dependencies —

easier to understand, easier to lazy-load, better tree-shaking, smoother onboarding."

Q: How do you bootstrap a standalone app?

" bootstrapApplication(RootComponent, appConfig) in main.ts . The appConfig holds app-level providers like provideRouter , provideHttpClient . No AppModule ."

Topic 13 — Signals

Signals are Angular's modern **reactivity primitive** — a value that auto-notifies dependents when it changes. Introduced in Angular 16, stabilized in 17. **Common interview topic in 2026.**

1. The Problem They Solve

Before signals:

```
export class Counter {  
  count = 0;  
  increment() { this.count++; }  
}
```

Works, but Angular doesn't really know `count` changed. Relies on Zone.js + full-tree change detection.

With signals:

```
export class Counter {  
  count = signal(0);  
  increment() { this.count.update(c => c + 1); }  
}
```

Angular knows exactly what depends on `count` . Only those re-render. **Fine-grained reactivity.**

2. Creating and Reading

```
import { signal } from '@angular/core';

count = signal(0);
name = signal('Ravi');
items = signal<string[]>([]);

// Read by CALLING it
console.log(this.count()); // 0
console.log(this.name()); // 'Ravi'
```

Key gotcha: Read with `()` . `count` is the signal; `count()` is the value.

In templates:

```
<p>Count: {{ count() }}</p>
```

3. Updating — Three Methods

```
count = signal(0);

this.count.set(10); // replace
this.count.update(c => c + 1); // derive from current

// For arrays/objects, use immutable updates
items = signal<string[]>([]);
this.items.update(list => [...list, 'apple']); //
// this.items().push('apple'); // won't notify
```

Signals detect changes by **reference**. Mutating an object's inside doesn't trigger.

`.mutate()` was deprecated — stick with `set` and `update` + immutable patterns.

4. `computed()` — Derived Signals

```
firstName = signal('Ravi');
lastName = signal('Kumar');
```



```
fullName = computed(() => `${this.firstName()} ${this.lastName()}`);
```

Change `firstName` `fullName` updates. Read-only. **Cached** — only recomputes when inputs change. **Lazy** — only runs if something reads it.

```
items = signal<Item[]>([]);
remainingCount = computed(() => this.items().filter(i => !i.done).length);
total = computed(() => this.items().reduce((sum, i) => sum + i.price, 0));
```

Don't store derived state as its own signal. Compute it.

5. `effect()` — Side Effects

```
import { effect } from '@angular/core';

constructor() {
  effect(() => {
    console.log('count changed to', this.count());
    localStorage.setItem('count', String(this.count()));
  });
}
```

Runs when any read signal changes. Tracks dependencies automatically.

For side effects, not derived values. Don't write to signals from inside effects — usually that's a `computed()` instead.

You'll use effects rarely. Computed covers 90%.

6. Signals in Templates

```
<p>Count: {{ count() }}</p>
<button (click)="count.set(count() + 1)"></button>

@if (count() > 10) { <p>That's a lot!</p> }

@for (item of items(); track item.id) {
```

```
<div>{{ item.name }}</div>
}
```

Always include the `()` .

7. Signal Inputs/Outputs (Angular 17.1+)

```
import { input, output } from '@angular/core';

@Component({
  template: `<h3>{{ product().name }}</h3>`
})
export class ProductCard {
  product = input.required<Product>();
  badge = input<string>('NEW');
  buy = output<Product>();
}
```

Differences from `@Input` / `@Output` :

- Functions, not decorators
- Read input by calling it: `product()`
- Type-safer
- `input.required<T>()` enforces parent passes it

8. Signals vs RxJS Observables

	Signals	Observables
What	Value that changes	Stream of values over time
Read	<code>mySignal()</code> (sync)	<code>subscribe()</code> (async)
Initial value	Always has one	May or may not emit immediately
Operators	<code>computed()</code> , <code>effect()</code>	Huge RxJS library
Use for	Component state, UI	HTTP, user input streams, WebSockets
Auto-cleanup	Yes	No — must unsubscribe

Memory leaks	Hard to cause	Easy if you forget
--------------	---------------	--------------------

9. Interop: `toSignal()` and `toObservable()`

```
import { toSignal, toObservable } from '@angular/core/rxjs-interop';

// Observable → Signal
users = toSignal(this.http.get<User[]>('/api/users'), { initialValue: [] });

// Signal → Observable
search = signal("");
search$ = toObservable(this.search);
```

Modern pattern: signals for state, Observables for async, bridge with these.

10. Worked Example — Search

```
@Component({
  selector: 'app-user-search',
  imports: [FormsModule],
  template: `
    <input [(ngModel)]="search" placeholder="Search..." />
    <p>{{ filtered().length }} of {{ users().length }} users</p>
    @for (u of filtered(); track u.id) {
      <div>{{ u.name }} — {{ u.email }}</div>
    } @empty {
      <p>No matches.</p>
    }
  `,
})
export class UserSearch {
  private http = inject(HttpClient);

  search = signal("");
  users = toSignal(this.http.get<User[]>('/api/users'), { initialValue: [] });

  filtered = computed(() => {
    const term = this.search().toLowerCase();
```

```

return this.users().filter(u =>
  u.name.toLowerCase().includes(term) ||
  u.email.toLowerCase().includes(term)
);
});
}

```

No `ngOnInit`, no subscriptions, no cleanup. 20 lines, fully reactive.

11. Signals + `OnPush` = Free Performance

```

import { ChangeDetectionStrategy } from '@angular/core';

@Component({
  changeDetection: ChangeDetectionStrategy.OnPush
})

```

Angular re-checks the component only when:

- `@Input` reference changes
- Event fires inside
- **A signal the component reads changes**

Fine-grained, opt-out reactivity. Recommended for new code.

12. Zoneless Mode

Angular is moving to **drop Zone.js entirely**. Signals are the path.

```

providers: [provideExperimentalZonelessChangeDetection()]

```

Smaller bundle, more predictable CD. Signals become mandatory.

Junior level: know it exists.

Common Mistakes

1. **Forgetting () when reading** — `{{ count }}` shows `[Object Object]`.
2. **Mutating instead of replacing** — `items().push(x)` doesn't notify.

3. **Writing to a signal from inside `effect`** — anti-pattern; use `computed` .
4. **Storing derived values as signals** — use `computed` .
5. **Reading once and reusing** — `const c = this.count();` is a snapshot, not live.

Interview Soundbites

Q: What are Angular Signals?

"A reactive primitive from Angular 16. A signal holds a value, tracks readers, notifies them on change. Three APIs: `signal()` for writable state, `computed()` for derived, `effect()` for side effects. Foundation for fine-grained, zoneless change detection."

Q: Signals vs Observables?

"Signals for synchronous reactive state — UI state, derived computations, form values. Observables for async streams — HTTP, user input over time, WebSockets. Modern code uses signals for state, Observables for async, bridges with `toSignal()` ."

Q: Difference between `signal()` , `computed()` , `effect()` ?

"`signal()` — writable reactive value. `computed()` — derived, read-only, cached. `effect()` — runs side effects when signals change; for outside-Angular things like `localStorage`."

Mental Model

Three sentences:

1. `signal()` is a value; read with `mySignal()` , write with `set / update` .
2. `computed()` is a derived value — read-only, cached, recomputes on dependency change.
3. `effect()` runs side effects on signal change.

Plus: **whenever state depends on other state, reach for `computed()` .**

Topic 14 — RxJS Basics — Observables, Subjects, BehaviorSubject

RxJS handles **asynchronous streams of values**. Every HTTP request returns an Observable. Router exposes Observables. You can't avoid it.

Junior-level RxJS is narrow — ~5 concepts and ~5 operators cover 90%.

1. The Mental Model

An **Observable** is a **stream of values over time** — like an array that emits one at a time.

```
Array:    [1, 2, 3, 4]           (all values NOW)
Observable: --1----2--3-----4---|   (over TIME, then completes)
```

Examples:

- HTTP response — emits once, completes
- Button click stream — emits on each click
- WebSocket — emits on each message
- Timer — emits every N ms

All Observables, same API.

2. Promise vs Observable

	Promise	Observable
Number of values	One	Zero, one, or many
Lazy?	No — starts immediately	Yes — does nothing until subscribed
Cancellable?	No	Yes
Operators?	<code>.then</code> , <code>.catch</code>	100+

Two critical differences:

1. **Lazy** — `http.get(...)` returns an Observable but **doesn't fire** until you subscribe.
2. **Multi-value** — Promises resolve once; Observables can emit forever.

3. Creating Observables

Mostly from Angular APIs:

```
this.http.get<User[]>('/api/users'); // emits once, completes
this.router.events;                 // emits on URL change
this.form.valueChanges;             // emits on every change
```

For learning, RxJS creation helpers:

```
import { of, from, interval, fromEvent } from 'rxjs';

of(1, 2, 3);                // emits 1, 2, 3, completes
from([1, 2, 3]);            // from array
from(fetch('/api/users'));  // from Promise
interval(1000);             // every 1s
fromEvent(button, 'click'); // every click
```

4. Subscribing

```
import { of } from 'rxjs';

const numbers$ = of(1, 2, 3);

numbers$.subscribe({
  next: value => console.log('got', value),
  error: err => console.error('failed:', err),
  complete: () => console.log('done!')
});
// Logs: got 1, got 2, got 3, done!
```

Shorthand (just `next`):

```
numbers$.subscribe(value => console.log(value));
```

Naming convention: `$` suffix — `users$`, `count$`. Common but not required.

5. The `async` Pipe Way

Manual:

```
ngOnInit() {
  this.sub = this.http.get<User[]>('/api/users').subscribe(data => {
    this.users = data;
  });
}
```

```
ngOnDestroy() {  
  this.sub?.unsubscribe();  
}
```

Idiomatic Angular:

```
users$ = inject(HttpClient).get<User[]>('/api/users');
```

```
@for (u of users$ | async; track u.id) { <p>{{ u.name }}</p> }
```

Or modern:

```
users = toSignal(inject(HttpClient).get<User[]>('/api/users'), { initialValue: [] });
```

Always prefer one of these. Manual subscribe only when imperatively needed.

6. Unsubscribing — Memory Leaks

Forgetting to unsubscribe keeps the component alive in memory.

async pipe (preferred)

```
{{ data$ | async }}
```

takeUntilDestroyed() (modern)

```
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';  
  
constructor() {  
  this.http.get<User[]>('/api/users')  
    .pipe(takeUntilDestroyed())  
    .subscribe(data => this.users = data);  
}
```

Manual Subscription


```
private sub = new Subscription();

ngOnInit() {
  this.sub.add(this.http.get(...).subscribe(...));
}

ngOnDestroy() {
  this.sub.unsubscribe();
}
```

7. Subject — Manually-Pushable

A `Subject` is both an Observable and an emitter.

```
import { Subject } from 'rxjs';

const clicks$ = new Subject<void>();

clicks$.subscribe(() => console.log('clicked A'));
clicks$.subscribe(() => console.log('clicked B'));

clicks$.next(); // logs "A", "B"
```

Late subscribers miss past emissions. Live stream.

Use for: custom events between services/components when you need notifications, not state.

```
@Injectable({ providedIn: 'root' })
export class NotificationService {
  private events$ = new Subject<string>();
  notifications$ = this.events$.asObservable();

  notify(msg: string) { this.events$.next(msg); }
}
```

8. BehaviorSubject — Subject With Current Value

```
import { BehaviorSubject } from 'rxjs';

const count$ = new BehaviorSubject<number>(0);

count$.subscribe(v => console.log('A got', v)); // "A got 0" immediately
count$.next(1);                                // "A got 1"
count$.subscribe(v => console.log('B got', v)); // "B got 1" (current value)
count$.next(2);                                // "A got 2", "B got 2"
```

Read sync: `count$.value` .

Use for: shared state with a current value. Canonical pre-signals pattern.

```
@Injectable({ providedIn: 'root' })
export class CartService {
  private _items$ = new BehaviorSubject<CartItem[]>([]);
  items$ = this._items$.asObservable();

  add(item: CartItem) {
    this._items$.next([...this._items$.value, item]);
  }
}
```

9. Subject vs BehaviorSubject

Use case	Choose
One-off events	Subject
Shared state with current value	BehaviorSubject
Late subscriber sees current state?	BehaviorSubject
Late subscriber misses past?	Subject

10. Cold vs Hot Observables

- **Cold:** each subscriber gets its own execution. **HTTP is cold** — subscribing twice = two requests.

- **Hot:** subscribers share execution. Subjects are hot.

Fix for repeated HTTP: `shareReplay()` , store the result, or `toSignal()` .

Common Mistakes

1. **Not subscribing.** HTTP doesn't fire. Use `.subscribe()` , `async` pipe, or `toSignal()` .
2. **Not unsubscribing.** Memory leak.
3. **Mutating BehaviorSubject array.** Use `.next([...this._items$.value, item])` .
4. **Multiple subscriptions to HTTP Observable.** Two requests. Use `shareReplay(1)` or `toSignal()` .
5. **Nested subscribes** — code smell. Use operators like `switchMap` .
6. **Confusing Subject and BehaviorSubject** — late subscriber behavior differs.

Interview Soundbites

Q: What's an Observable?

"A stream of values over time, from RxJS. Lazy — doesn't run until subscribed. Cancellable. Every HTTP request returns one. Consume via `.subscribe()` , `async` pipe, or `toSignal()` ."

Q: Observable vs Promise?

"Promises emit one value, eagerly. Observables emit any number, lazily, cancellable, with a rich operator library. Promises for one-shot async; Observables for streams."

Q: Subject vs BehaviorSubject?

"Both Observables you push values into. `Subject` doesn't hold current value — new subscribers don't get past emissions. `BehaviorSubject` requires initial value, emits current to new subscribers. Use `BehaviorSubject` for shared state, `Subject` for events."

Q: How do you avoid memory leaks from subscriptions?

"`async` pipe in templates. `takeUntilDestroyed()` for manual subscribes. Manual `Subscription.unsubscribe()` in `ngOnDestroy` as fallback."

Mental Model

Five sentences:

1. **An Observable is a stream of values over time.**
2. **Lazy — `.subscribe()` makes it run.**

3. `async pipe` and `toSignal()` are safe — auto-clean up.
 4. `Subject` is a manual emitter; subscribers get future emissions.
 5. `BehaviorSubject` is a `Subject` with current value; new subscribers get it immediately.
-

Topic 15 — RxJS Operators

Operators transform Observables. RxJS has 100+. **You'll use about 10.**

1. The `pipe()` Method

```
import { map, filter } from 'rxjs';

source$.pipe(
  filter(x => x > 5),
  map(x => x * 2)
).subscribe(value => console.log(value));
```

Read left-to-right. Each operator returns a new Observable.

2. `map` — Transform Each Value

```
import { of, map } from 'rxjs';

of(1, 2, 3).pipe(
  map(x => x * 10)
).subscribe(v => console.log(v));
// 10, 20, 30
```

Real example:

```
this.http.get<{ data: User[] }>('/api/users').pipe(
  map(response => response.data)
).subscribe(users => /* ... */);
```

3. filter — Drop Values

```
of(1, 2, 3, 4, 5).pipe(
  filter(x => x % 2 === 0)
).subscribe(v => console.log(v));
// 2, 4
```

```
this.router.events.pipe(
  filter(event => event instanceof NavigationEnd)
).subscribe(/* ... */);
```

4. tap — Side Effects (Debugging)

```
of(1, 2, 3).pipe(
  tap(x => console.log('before:', x)),
  map(x => x * 2),
  tap(x => console.log('after:', x))
).subscribe();
```

Doesn't change the value. For logging/debugging.

5. switchMap — Switch to a New Inner Observable

The most important flattening operator. Guaranteed interview question.

When each outer value should trigger an async operation, and you only care about the **latest**:

```
inputElement.valueChanges.pipe(
  switchMap(query => this.http.get<Result[]>(`/api/search?q=${query}`))
).subscribe(results => /* ... */);
```

Behavior:

1. For each outer value, calls a function returning an inner Observable.
2. Subscribes to it, re-emits its values.
3. **New outer value before inner completes** **cancels previous inner.**

Search-as-you-type: user types “a” request; types “ab” 200ms later first request cancelled, new request fires. Prevents race conditions where old response overwrites new.

6. mergeMap , concatMap , exhaustMap

Same family, different “what about previous” behavior:

Operator	When new outer arrives
switchMap	Cancel previous
mergeMap	Run all in parallel
concatMap	Queue — finish then start
exhaustMap	Ignore until current finishes

- **switchMap** — search, refetch on input. Most common.
- **mergeMap** — independent parallel work (analytics events).
- **concatMap** — order matters (sequential logs).
- **exhaustMap** — prevent duplicate submissions.

Know **switchMap** deeply, recognize the others.

7. debounceTime — Wait for Silence

```
searchInput.valueChanges.pipe(  
  debounceTime(300),  
  switchMap(q => this.http.get(`/api/search?q=${q}`))  
)
```

User types “react” — without debounce: 5 requests. With **debounceTime(300)** : 1 request after typing pauses.

Pair with **distinctUntilChanged()** to skip same-as-previous values:

```
.pipe(  
  debounceTime(300),  
  distinctUntilChanged(),  
)
```

```
switchMap(q => /* ... */)
)
```

8. **take** / **takeUntil** / **takeUntilDestroyed**

take(n)

```
interval(1000).pipe(take(5)).subscribe();
// 0, 1, 2, 3, 4, completes
```

take(1) for “first value only.”

takeUntil(notifier\$)

```
private destroy$ = new Subject<void>();

ngOnInit() {
  interval(1000).pipe(takeUntil(this.destroy$)).subscribe();
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
```

Pre-Angular 16 cleanup pattern.

takeUntilDestroyed() — modern

```
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

constructor() {
  interval(1000).pipe(takeUntilDestroyed()).subscribe();
}
```

Use this in new code.

9. `catchError` — Recover from Errors

```
import { catchError, of } from 'rxjs';

this.http.get<User[]>('/api/users').pipe(
  catchError(err => {
    console.error(err);
    return of([]); // fallback to empty
  })
).subscribe(users => /* ... */);
```

Without it, error completes the Observable. With it, you substitute a fallback.

Almost every HTTP call should have `catchError` somewhere.

10. `combineLatest` — Combine Streams

```
import { combineLatest, map } from 'rxjs';

const filters$ = combineLatest([
  this.searchTerm$,
  this.category$,
  this.sortOrder$
]).pipe(
  map(([term, cat, sort]) => ({ term, cat, sort }))
);
```

Waits for each source to emit at least once, then emits on every subsequent emission.

11. `shareReplay` — Cache for Multiple Subscribers

```
users$ = this.http.get<User[]>('/api/users').pipe(
  shareReplay({ bufferSize: 1, refCount: true })
);
```

First subscriber fires request. Subsequent get cached value.

12. The Five You MUST Know

Operator	Purpose
<code>map</code>	Transform each value
<code>filter</code>	Drop non-matching
<code>switchMap</code>	Switch to inner, cancel previous
<code>debounceTime</code>	Wait for silence
<code>takeUntilDestroyed</code>	Auto-clean on destroy

13. Realistic Search Example

```
@Component({
  imports: [FormsModule],
  template: `
    <input [(ngModel)]="query" (ngModelChange)="onSearch($event)" />
    @if (loading()) { <p>Searching...</p> }
    @for (user of users(); track user.id) {
      <div>{{ user.name }}</div>
    }
  `,
})
export class UserSearch {
  private http = inject(HttpClient);
  query = "";
  loading = signal(false);

  private searchTerm$ = new Subject<string>();

  users = toSignal(
    this.searchTerm$.pipe(
      debounceTime(300),
      distinctUntilChanged(),
      filter(q => q.length >= 2),
      switchMap(q => {
        this.loading.set(true);
        return this.http.get<User[]>(`/api/users?q=${q}`).pipe(
```

```

        catchError(() => of([] as User[]))
    );
  },
  map(results => {
    this.loading.set(false);
    return results;
  }),
  takeUntilDestroyed()
),
{ initialValue: [] as User[] }
);

onSearch(value: string) { this.searchTerm$.next(value); }
}

```

50+ lines of imperative code 1 declarative pipeline. **This is what RxJS is for.**

Common Mistakes

1. **Forgetting `.pipe()`** — `source$.map(...)` errors.
2. **`map` instead of `switchMap` for `async`** — produces a stream of Observables.
3. **Nested subscribes** — code smell. Use `switchMap` .
4. **`catchError` in wrong place** — usually want it INSIDE `switchMap` so outer pipeline keeps going.
5. **Side effects in `map`** — use `tap` .
6. **Missing `distinctUntilChanged` after `debounceTime`** — re-fires for unchanged values.

Interview Soundbites

Q: What does `switchMap` do?

"Maps each outer value to a new inner Observable, subscribes, re-emits values. Key behavior: new outer value cancels previous inner subscription. Perfect for search-as-you-type — prevents race conditions where older responses arrive after newer ones."

Q: Difference between `switchMap` , `mergeMap` , `concatMap` ?

"All flatten outer Observables into inner. `switchMap` cancels previous (search). `mergeMap` runs all in parallel (independent work). `concatMap` queues (order matters)."

Q: How do you build a debounced search?

"Pipe input through `debounceTime(300)`, `distinctUntilChanged`, `switchMap` into HTTP call. Wrap HTTP with `catchError` for failure. Add `takeUntilDestroyed()` for cleanup."

Mental Model

Five sentences:

1. `.pipe()` applies operators; they return new Observables.
 2. `map` / `filter` / `tap` transform/drop/peek.
 3. `switchMap` for async-per-emission, canceling previous. Workhorse for HTTP-on-input.
 4. `debounceTime` + `distinctUntilChanged` + `switchMap` = canonical search.
 5. `catchError` recovers; `takeUntilDestroyed` auto-cleans.
-

Topic 16 — HttpClient

Angular's built-in service for HTTP. Returns Observables, typed responses, plays with interceptors.

1. Setup

```
// app.config.ts
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(),
    provideRouter(routes)
  ]
};
```

That's it. (Old `HttpClientModule` is deprecated.)

2. Inject and Call

```
import { HttpClient } from '@angular/common/http';

@Component({ /* ... */ })
export class UserList {
  private http = inject(HttpClient);

  loadUsers() {
    this.http.get<User[]>('/api/users').subscribe(users => {
      console.log(users);
    });
  }
}
```

The `<User[]>` generic types the response.

3. The Five Methods

```
this.http.get<User[]>('/api/users');
this.http.post<User>('/api/users', { name: 'Ravi' });
this.http.put<User>('/api/users/1', { id: 1, name: 'Ravi' });
this.http.patch<User>('/api/users/1', { name: 'Ravi K' });
this.http.delete<void>('/api/users/1');
```

Angular auto-serializes objects to JSON and sets `Content-Type: application/json` for `POST` / `PUT` / `PATCH` .

4. Query Parameters

Object form

```
this.http.get<User[]>('/api/users', {
  params: { status: 'active', limit: 10, page: 2 }
});
```

HttpParams (immutable)

```
import { HttpParams } from '@angular/common/http';

const params = new HttpParams()
  .set('status', 'active')
  .set('limit', '10');

this.http.get<User[]>('/api/users', { params });
```

Important: Immutable. `.set()` returns a new instance:

```
let p = new HttpParams();
p.set('a', '1');    // no effect
p = p.set('a', '1'); // reassign
```

For repeated params (`?tag=js&tag=ts`):

```
.append('tag', 'js').append('tag', 'ts');
```

5. Custom Headers

```
import { HttpHeaders } from '@angular/common/http';

const headers = new HttpHeaders({
  'Authorization': 'Bearer abc123',
  'X-Custom-Header': 'value'
});

this.http.get<User[]>('/api/users', { headers });
```

Also immutable. **Don't add `Authorization` manually everywhere — use an interceptor (topic 29).**

6. Reading Full Response

```
this.http.get<User[]>('/api/users', { observe: 'response' })
  .subscribe(response => {
```

```
console.log(response.status);
console.log(response.headers.get('X-Total-Count'));
console.log(response.body);
});
```

7. Error Handling

Inline

```
this.http.get<User[]>('/api/users').subscribe({
  next: users => this.users = users,
  error: err => {
    console.error(err);
    this.errorMessage = 'Could not load';
  }
});
```

Operator-level (preferred)

```
import { catchError, of } from 'rxjs';

this.http.get<User[]>('/api/users').pipe(
  catchError(err => {
    console.error(err);
    return of([]);
  })
).subscribe(users => this.users = users);
```

Error is `HttpErrorResponse` with `status` , `statusText` , `error` , `url` :

```
catchError((err: HttpErrorResponse) => {
  if (err.status === 401) this.router.navigate(['/login']);
  return of([]);
})
```

For app-wide handling: use an interceptor (topic 29).

8. Standard Service Shape

```
@Injectable({ providedIn: 'root' })
export class UserService {
  private http = inject(HttpClient);
  private base = '/api/users';

  list(filters: { status?: string; limit?: number } = {}): Observable<User[]> {
    let params = new HttpParams();
    if (filters.status) params = params.set('status', filters.status);
    if (filters.limit) params = params.set('limit', String(filters.limit));
    return this.http.get<User[]>(this.base, { params });
  }

  getById(id: number) { return this.http.get<User>(`${this.base}/${id}`); }
  create(data: Omit<User, 'id'>) { return this.http.post<User>(this.base, data); }
  update(id: number, data: Partial<User>) { return this.http.patch<User>(`${this.base}/${id}`, data); }
  delete(id: number) { return this.http.delete<void>(`${this.base}/${id}`); }
}
```

Component:

```
@Component({
  template: `@for (u of users(); track u.id) { <p>{{ u.name }}</p> }`
})
export class UserList {
  private userService = inject(UserService);
  users = toSignal(this.userService.list(), { initialValue: [] });
}
```

Five lines of component. HTTP details in service.

9. Template Consumption

async pipe

```
users$ = this.userService.list();
```

```
@for (u of users$ | async; track u.id) { ... }
```

toSignal() (modern)

```
users = toSignal(this.userService.list(), { initialValue: [] });
```

```
@for (u of users(); track u.id) { ... }
```

Both auto-subscribe and auto-clean. Pick one per component.

10. POST + Form-style Requests

Form-encoded

```
const body = new HttpParams().set('username', 'ravi').set('password', 'secret');
this.http.post('/api/login', body.toString(), {
  headers: { 'Content-Type': 'application/x-www-form-urlencoded' }
});
```

File upload

```
const formData = new FormData();
formData.append('avatar', file);
formData.append('name', 'Ravi');

this.http.post('/api/upload', formData);
// Don't set Content-Type — browser sets it with boundary
```

11. Cold Observable Gotcha

Every HTTP Observable is cold. Each subscriber = new request.

```
const users$ = this.http.get<User[]>('/api/users');
users$.subscribe(/* ... */); // fires
```



```
users$.subscribe(/* ... */); // fires AGAIN
```

Two `async` pipes on same Observable → two HTTP requests.

Fixes:

- Subscribe once, store result
- `shareReplay(1)`
- `toSignal()` (caches automatically)

12. Interceptors Preview

A function that runs before every request goes out. Use for:

- Authorization header on every request
- Logging
- Global loader
- Catching 401s → login redirect
- Retries

Topic 29 covers fully.

Common Mistakes

1. **Forgetting to subscribe** — `http.get(...)` alone does nothing.
2. **Not typing response** — `http.get('/api/users')` is `Observable<unknown>`.
3. **Mutating `HttpParams` / `HttpHeaders`** — immutable, must reassign.
4. **Setting `Content-Type` for `FormData`** — breaks uploads (browser sets it).
5. **Multiple `async` pipes** — multiple network calls.
6. **Subscribing inside services** — services return Observables; components subscribe.
7. **Business logic in components** — move to service methods.

Interview Soundbites

Q: How do you make HTTP calls?

"Inject `HttpClient`, call `.get/.post/.put/.patch/.delete`. Each returns an `Observable`. Set up with `provideHttpClient()` in `app.config.ts`. Consume via `async` pipe, `toSignal()`, or manual subscription with cleanup."

Q: How do you type the response?

“Generic type parameter: `this.http.get<User[]>('/api/users')` . TypeScript assertion, not runtime validation.”

Q: Why does my API get called twice?

“HTTP Observables are cold — every subscriber triggers a request. Two `async` pipes = two calls. Fix with `shareReplay(1)` , `toSignal()` , or subscribe once and store.”

Q: How do you add auth header to every request?

“HTTP interceptor — register with `withInterceptors([authInterceptor])` in `provideHttpClient` . Centralizes the logic.”

Mental Model

Five sentences:

1. `provideHttpClient()` , then inject `HttpClient` , call `.get/.post/.put/.patch/.delete` .
 2. Type with generics: `http.get<User[]>('/api/users')` .
 3. `HttpParams` / `HttpHeaders` are immutable; chain `.set()` .
 4. Consume via `async` pipe or `toSignal()` ; services return Observables.
 5. Cross-cutting concerns (auth, logging, errors) interceptors.
-

Topic 17 — The `async` Pipe

Subscribes to an Observable (or Promise) in the template, returns its latest value, auto-unsubscribes on destroy. Most-used pipe in Angular.

1. Basic Usage

```
@Component({
  imports: [AsyncPipe],
  template: `
    @for (u of users$ | async; track u.id) {
      <p>{{ u.name }}</p>
    }
  `,
})
```

```
export class UserList {  
  private http = inject(HttpClient);  
  users$ = this.http.get<User[]>('/api/users');  
}
```

Behind the scenes:

1. Subscribes to `users$` on render.
2. Returns latest value on each emission.
3. Auto-unsubscribes on destroy.

2. Why It Exists

Without:

```
export class UserList implements OnInit, OnDestroy {  
  users: User[] = [];  
  private sub?: Subscription;  
  
  ngOnInit() {  
    this.sub = this.http.get<User[]>('/api/users')  
      .subscribe(users => this.users = users);  
  }  
  
  ngOnDestroy() {  
    this.sub?.unsubscribe();  
  }  
}
```

With:

```
export class UserList {  
  users$ = inject(HttpClient).get<User[]>('/api/users');  
}
```

Absorbs subscription, value extraction, unsubscription, and change detection into one declarative pipe.

3. With Promises Too

```
user = fetch('/api/me').then(r => r.json());
```

```
{{ user | async }}
```

Mostly used with Observables.

4. The `as` Clause

When you need the value in multiple places, **don't pipe multiple times**:

Wrong (subscribes 3 times):

```
@if (user$ | async) {  
  <h2>{{ (user$ | async)?.name }}</h2>  
  <p>{{ (user$ | async)?.email }}</p>  
}
```

Right:

```
@if (user$ | async; as user) {  
  <h2>{{ user.name }}</h2>  
  <p>{{ user.email }}</p>  
}
```

`as user` stores the resolved value in a template variable. One subscription.

5. Two Subscriptions Trap

```
@if (users$ | async; as users) {  
  <p>Total: {{ users.length }}</p>  
}  
@for (u of users$ | async; track u.id) {  
  <p>{{ u.name }}</p>  
}
```

Two `| async` = two subscriptions = two HTTP calls.

Fixes:

- One `async` at top level with `as` :

```
@if (users$ | async; as users) {  
  <p>Total: {{ users.length }}</p>  
  @for (u of users; track u.id) { <p>{{ u.name }}</p> }  
}
```

- `shareReplay(1)` on Observable
- `toSignal()` instead

6. Initial Value Is `null`

Until first emission, `async` returns `null` :

```
<p>{{ (user$ | async)?.name }}</p> <!-- safe with ?. -->
```

For HTTP, show loading:

```
@if (users$ | async; as users) {  
  @for (u of users; track u.id) { <p>{{ u.name }}</p> }  
} @else {  
  <p>Loading...</p>  
}
```

7. Change Detection

When Observable emits, `async` does two things:

1. Returns new value.
2. Calls `markForCheck()` — telling Angular to re-render.

This is why **`async` pipe + `OnPush`** works — Angular knows to update `OnPush` components.

8. `async` Pipe vs `toSignal()`

	async pipe	toSignal()
Where	Template	Component class
Read in template	obs\$ async	mySignal()
Initial value	null	Configurable
Auto-unsubscribes	Yes	Yes
Reuse	as user local	Read repeatedly
Multiple usages	Risk multi-sub	Cached
Works with OnPush	Yes	Yes (better with signals)

New code: prefer `toSignal()` . Existing: `async` is fine.

9. Realistic Example — Master-Detail With States

```

type State<T> =
  | { kind: 'loading' }
  | { kind: 'success'; data: T }
  | { kind: 'error'; message: string };

@Component({
  imports: [AsyncPipe],
  template: `
    @if (state$ | async; as state) {
      @switch (state.kind) {
        @case ('loading') { <p>Loading...</p> }
        @case ('error') { <p>{{ state.message }}</p> }
        @case ('success') {
          @for (u of state.data; track u.id) {
            <div>{{ u.name }}</div>
          } @empty {
            <p>No users.</p>
          }
        }
      }
    }
  `
})

```

```

})
export class UserList {
  private http = inject(HttpClient);

  state$ = this.http.get<User[]>('/api/users').pipe(
    map(users => ({ kind: 'success', data: users } as State<User[]>)),
    catchError(err => of({ kind: 'error', message: err.message } as State<User[]>)),
    startWith({ kind: 'loading' } as State<User[]>)
  );
}

```

Common Mistakes

1. Forgetting to import `AsyncPipe` .
2. Multiple `| async` on same `Observable`.
3. Not handling `null` initial.
4. Confusing with JS `async` / `await` — different things.

Interview Soundbites

Q: What is the `async` pipe?

"Built-in pipe that subscribes to an Observable/Promise in the template, returns latest value, auto-unsubscribes on destroy. Preferred way to consume Observables in templates — no manual subscription management."

Q: What happens with `| async` twice on same Observable?

"Each use is a separate subscription. For cold Observables like HTTP, that's two calls. Fix with `as` local variable, `shareReplay(1)` , or `toSignal()` ."

Q: `async` pipe or `toSignal()` ?

"Both correct. `async` is classic. `toSignal()` is modern signal-based equivalent — better for multi-read scenarios, typed initial value, signal ecosystem integration."

Mental Model

Four sentences:

1. `| async` subscribes, extracts value, unsubscribes on destroy.
2. Initial value is `null` ; use `?.` or `@if (... as value)` .

3. Each `| async` = separate subscription — share via `as` , `shareReplay(1)` , `toSignal()` .
 4. Triggers CD on emission — works perfectly with `OnPush` .
-

Topic 18 — Routing — The Basics

In a SPA, there's no real page reload. The Router maps URLs to components, uses `<router-outlet>` as the rendering slot, and `routerLink` for navigation without reload.

1. Setup

```
// app.routes.ts
import { Routes } from '@angular/router';

export const routes: Routes = [];
```

```
// app.config.ts
import { provideRouter } from '@angular/router';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes)]
};
```

```
<!-- app.html -->
<header>...</header>
<router-outlet></router-outlet>
<footer>...</footer>
```

```
// app.ts
import { RouterOutlet } from '@angular/router';

@Component({
  imports: [RouterOutlet],
  templateUrl: './app.html'
```



```
})  
export class App {}
```

2. Defining Routes

```
import { Home } from './pages/home';  
import { About } from './pages/about';  
import { NotFound } from './pages/not-found';  
  
export const routes: Routes = [  
  { path: '', component: Home },  
  { path: 'about', component: About },  
  { path: 'users', component: UserList },  
  { path: '**', component: NotFound } // wildcard, MUST be last  
];
```

- path: '' /
- path: 'about' /about
- path: '**' catch-all (404). Must be last; routes match top-to-bottom.

No leading slash on path . path: 'about' , not path: '/about' .

3. routerLink

```
<a routerLink="/about">About</a>  
<a [routerLink]="['/users', userId]">View user {{ userId }}</a>
```

Component must import:

```
import { RouterLink } from '@angular/router';  
  
@Component({ imports: [RouterLink] })
```

Never use href for in-app links — causes full page reload.

4. routerLinkActive

```
<a routerLink="/about" routerLinkActive="active">About</a>
```

Adds the `active` class when route matches.

By default, prefix match. For exact match (home link):

```
<a routerLink="/" routerLinkActive="active" [routerLinkActiveOptions]="{exact: true}">H
```

Import: `RouterLinkActive` .

5. Programmatic Navigation

```
import { Router } from '@angular/router';

@Component({ /* ... */ })
export class LoginForm {
  private router = inject(Router);

  onLoginSuccess() {
    this.router.navigate(['/dashboard']);
  }

  viewUser(id: number) {
    this.router.navigate(['/users', id]);
  }

  search(term: string) {
    this.router.navigate(['/search'], { queryParams: { q: term } });
  }
}
```

`router.navigate(['/users', id])` is the array form. `router.navigateByUrl('/users/42')` takes raw URL.

Returns `Promise<boolean>` :

```
this.router.navigate(['/admin']).then(success => {
  if (!success) { /* guard blocked */ }
```

```
});
```

6. Nested Routes (Children)

```
export const routes: Routes = [
  {
    path: 'users',
    component: UsersLayout,
    children: [
      { path: '', component: UserList }, // /users
      { path: 'new', component: UserCreate }, // /users/new
      { path: ':id', component: UserDetail }, // /users/42
    ]
  }
];
```

Parent template needs its own outlet:

```
<h2>Users section</h2>
<nav>
  <a routerLink="">List</a>
  <a routerLink="new">New</a>
</nav>
<router-outlet></router-outlet>
```

Child `routerLink` s **without leading slash** = relative to parent.

7. Redirects

```
export const routes: Routes = [
  { path: '', redirectTo: '/home', pathMatch: 'full' },
  { path: 'home', component: Home }
];
```

`pathMatch: 'full'` required for empty-path redirects — otherwise infinite loop.

8. Route Matching Order

Top-to-bottom, first match wins. **Order matters.**

```
// Wildcard catches everything
{ path: '**', component: NotFound },
{ path: 'about', component: About } // never reached
```

```
// Specific dynamic wildcard
{ path: 'about', component: About },
{ path: '**', component: NotFound }
```

Same for `users/new` before `users/:id`.

9. Router vs ActivatedRoute

Service	What
Router	Imperative: <code>navigate</code> , listen to nav events
ActivatedRoute	Read current route info — params, query params, data

`Router` = action. `ActivatedRoute` = state.

10. Hash vs Path Routing

Default: path-based (`/users/42`). Requires server config (serve `index.html` for unmatched).

Hash-based (`/#/users/42`) — no server config needed:

```
import { provideRouter, withHashLocation } from '@angular/router';
provideRouter(routes, withHashLocation())
```

Almost always use path routing. Hash only when you can't configure server (GitHub Pages).

Common Mistakes

1. **Forgetting `<router-outlet>`** — routes match silently, nothing renders.
2. **Forgetting to import `RouterOutlet` / `RouterLink` / `RouterLinkActive`** .
3. **Using `href` instead of `routerLink`** — full page reload.
4. **Wildcard before specific routes.**
5. **Missing `pathMatch: 'full'`** on empty-path redirect — infinite loop.
6. **Leading slashes in `path`** .

Interview Soundbites

Q: How does routing work?

"Array of routes mapping paths to components, registered with `provideRouter(routes)` . `<router-outlet>` is the slot where matched component renders. Navigation via `routerLink` or `router.navigate()` . Page never reloads — Angular swaps components."

Q: `Router` vs `ActivatedRoute` ?

" `Router` for imperative — `navigate` , listen to events. `ActivatedRoute` for reading current route — `params`, `query params`, `data`. `Router` to act; `ActivatedRoute` to read."

Q: How handle 404?

"Wildcard route at bottom: `{ path: '**', component: NotFound }` . First match wins, so it catches anything unmatched."

Mental Model

Five sentences:

1. **Routes = array mapping paths to components, registered with `provideRouter`** .
 2. **`<router-outlet>` is the slot; nav/footer stay put outside it.**
 3. **Use `routerLink` in templates, `router.navigate(...)` in code — never `href`** .
 4. **Inject `Router` to act; `ActivatedRoute` to read.**
 5. **Order routes specific dynamic wildcard, top-to-bottom, first match wins.**
-

Topic 19 — Route Params & Query Params

Real apps need URLs that carry data. `/users/42` should fetch user 42. `/search?q=react` should filter.

1. Route Params vs Query Params

	Route param	Query param
Position	Part of path	After <code>?</code>
Example	<code>/users/42</code>	<code>/search?q=react</code>
Defined in route	<code>path: 'users/:id'</code>	Not defined
When	<i>Which</i> resource	<i>How</i> you're viewing

2. Defining Route Params

```
export const routes: Routes = [  
  { path: 'users',      component: UserList },  
  { path: 'users/:id',  component: UserDetail }, // /users/42  
  { path: 'shop/:cat/:id', component: Product }   // /shop/books/123  
];
```

Colon = variable. **Captured as string** — even numeric-looking.

Query params not defined in routes.

3. Reading Route Params — Two Approaches

Critical: get this wrong, stale data forever.

```
import { ActivatedRoute } from '@angular/router';  
  
@Component({ /* ... */ })  
export class UserDetail {  
  private route = inject(ActivatedRoute);  
}
```

A. `snapshot` (synchronous, one-shot)

```
const id = this.route.snapshot.paramMap.get('id');
```

Does NOT update if URL changes. Use only when:

- Read once at init
- Component destroyed/recreated on URL change

B. `paramMap` Observable (reactive, recommended)

```
this.route.paramMap.subscribe(params => {  
  const id = params.get('id');  
  this.loadUser(id);  
});
```

Emits on every param change (including initial). **Safer default.**

Idiomatic Pattern

```
@Component({ /* ... */ })  
export class UserDetail {  
  private route = inject(ActivatedRoute);  
  private userService = inject(UserService);  
  
  user = toSignal(  
    this.route.paramMap.pipe(  
      switchMap(params => this.userService.getById(Number(params.get('id'))))  
    )  
  );  
}
```

`switchMap` refetches on URL change, cancels previous. Bridges to signal.

4. The “Same Component, New URL” Gotcha

Angular **reuses** component instance when only param changes within same route:

```
/users/1    UserDetail created, ngOnInit runs
```

/users/2 SAME instance, ngOnInit does NOT run

If you used `snapshot`, you stay on user 1. **Always use `paramMap` `Observable`** for params that can change.

5. Query Params

Same pattern, `queryParamMap` :

```
this.route.queryParamMap.subscribe(params => {  
  const q = params.get('q');  
  const page = params.get('page');  
});
```

Multi-value: `params.getAll('tag')` `['js', 'ts']` for `?tag=js&tag=ts` .

6. Setting Query Params

```
private router = inject(Router);  
  
// /search?q=react&page=2  
this.router.navigate(['/search'], {  
  queryParams: { q: 'react', page: 2 }  
});  
  
// Merge with existing  
this.router.navigate([], {  
  queryParams: { sort: 'newest' },  
  queryParamsHandling: 'merge' // keep existing  
});
```

`queryParamsHandling` :

- *omitted* — replaces all
- `'merge'` — keep existing, add/replace specified (90% of filter UIs)
- `'preserve'` — keep all, ignore new

Templates:


```
<a [routerLink]="['/search']" [queryParams]="{ q: 'react' }">Search</a>
<a [routerLink]="[]" [queryParams]="{ page: 2 }" queryParamsHandling="merge">Page 2
```

7. Worked Example — Search with URL State

```
@Component({
  imports: [FormsModule, RouterLink],
  template: `
    <input
      [(ngModel)]="searchTerm"
      (ngModelChange)="onSearchChange($event)"
      placeholder="Search..." />

    <select [(ngModel)]="sortOrder" (ngModelChange)="onSortChange($event)">
      <option value="name">Name</option>
      <option value="price">Price</option>
    </select>

    @for (p of products(); track p.id) {
      <a [routerLink]="['/products', p.id]">{{ p.name }} — {{ p.price }}</a>
    } @empty {
      <p>No matches.</p>
    }
  `,
})
export class Search {
  private route = inject(ActivatedRoute);
  private router = inject(Router);
  private productService = inject(ProductService);

  searchTerm = "";
  sortOrder = 'name';

  products = toSignal(
    this.route.queryParamMap.pipe(
      map(params => ({
        q: params.get('q') ?? "",
        sort: params.get('sort') ?? 'name'
      }
    )
  )
}
```

```

    })),
    map(filters => {
      this.searchTerm = filters.q;
      this.sortOrder = filters.sort;
      return filters;
    }),
    switchMap(({ q, sort }) => this.productService.search(q, sort))
  ),
  { initialValue: [] }
);

onSearchChange(q: string) {
  this.router.navigate([], {
    queryParams: { q: q || null },
    queryParamsHandling: 'merge'
  });
}

onSortChange(sort: string) {
  this.router.navigate([], {
    queryParams: { sort },
    queryParamsHandling: 'merge'
  });
}
}

```

Free wins:

- Refresh keeps state
- Back button works
- URLs are shareable
- Two-way sync (type URL, back input)

`queryParams: { q: q || null }` — `null` removes the param.

8. `params` vs `paramMap`

```

// Old, less common
this.route.params.subscribe(p => console.log(p['id']));

```

```
// Recommended
this.route.paramMap.subscribe(p => console.log(p.get('id')));
```

Use `paramMap` and `queryParams` .

9. Always Strings — Convert

```
const id = Number(params.get('id'));
const page = Number(params.get('page') ?? '1');
```

`Number(null)` is `0` — handle missing params.

10. Route data

Static data on routes:

```
{ path: 'admin', component: Admin, data: { title: 'Admin Panel' } }
```

```
const title = this.route.snapshot.data['title'];
```

Common Mistakes

1. **Using `snapshot` when URL can change** — stale data.
2. **`params['id']` old style** — use `paramMap.get('id')` .
3. **Forgetting params are strings** — convert with `Number()` .
4. **Missing `queryParamsHandling: 'merge'`** in filter UIs — wipes other params.
5. **Defining query params in route config** — you don't.
6. **Not handling missing params** — `null` cases.
7. **Subscribing without cleanup** — leaks.

Interview Soundbites

Q: Route params vs query params?

"Route params are path parts, defined with `:name` , identify which resource. Query params come after `?` , not defined in routes, modify how you view."

Q: How read a route parameter?

"Inject `ActivatedRoute` . Read via `snapshot.paramMap.get('id')` for one-time, or subscribe to `paramMap` `Observable` to react to changes. `Observable` safer — Angular reuses components when only params change."

Q: Why use `Observable` form over `snapshot`?

"Angular reuses component instances when navigating between matching routes like `/users/1` `/users/2` . `Constructor` and `ngOnInit` don't re-run. `Snapshot` stays stale. `paramMap` emits on every param change."

Q: How sync URL with UI state?

"Drive UI from URL, not the other way. Subscribe to `queryParams` , update displayed data. On interaction, `router.navigate([], { queryParams, queryParamsHandling: 'merge' })` . URL becomes single source of truth — refresh, back, share work for free."

Mental Model

Five sentences:

1. Route params (`:id`) are path; query params (`?q=...`) after `?` .
2. Inject `ActivatedRoute` ; read via `paramMap` (path) and `queryParams` (query).
3. Prefer `Observable` forms — components reuse, snapshot goes stale.
4. Combine `paramMap` with `switchMap` to refetch on URL change; bridge to signal with `toSignal` .
5. Store filter state in URL via `router.navigate([], { queryParams, queryParamsHandling: 'merge' })` .

Topic 20 — Route Guards

A **guard** is a function the router calls before letting navigation happen. Returns `true` (allow), `false` (block), or `UrlTree` (redirect).

1. The Five Guard Types

Guard	When	Common use
<code>canActivate</code>	Before navigating INTO	Auth check

<code>canActivateChild</code>	Before child of route	Same check for all children
<code>canDeactivate</code>	Before leaving	Unsaved changes prompt
<code>canMatch</code>	Before matching at all	Conditional route (feature flags, roles)
<code>resolve</code>	Before activating, to fetch	Pre-load data

Must-knows: `canActivate` and `canDeactivate` .

2. Functional Guard Pattern

Basic `canActivate`

```
// guards/auth.guard.ts
import { CanActivateFn, Router } from '@angular/router';
import { inject } from '@angular/core';
import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn()) {
    return true;
  }

  return router.createUrlTree(['/login']);
};
```

- `CanActivateFn` is the type
- `inject()` works inside guards
- Returns `true` / `false` / `UrlTree`

Wiring it

```
export const routes: Routes = [
  { path: 'login', component: Login },
```

```
{ path: 'dashboard', component: Dashboard, canActivate: [authGuard] }  
];
```

Array — stack guards. All must pass.

3. Asynchronous Guards

Return Observable or Promise:

```
export const authGuard: CanActivateFn = () => {  
  const auth = inject(AuthService);  
  const router = inject(Router);  
  
  return auth.verifyToken().pipe(  
    map(valid => valid ? true : router.createUrlTree(['/login'])),  
    catchError(() => of(router.createUrlTree(['/login'])))  
  );  
};
```

Always make Observable complete (use `take(1)`) or navigation hangs:

```
return auth.user$.pipe(  
  take(1),  
  map(user => user ? true : router.createUrlTree(['/login']))  
);
```

Classic bug: forgetting `take(1)` on a `BehaviorSubject` .

4. `canDeactivate` — Unsaved Changes

The guard asks the component itself:

```
export interface HasUnsavedChanges {  
  hasUnsavedChanges(): boolean;  
}  
  
export const unsavedChangesGuard: CanDeactivateFn<HasUnsavedChanges> =  
  (component) => {  
    if (!component.hasUnsavedChanges()) return true;
```

```
    return confirm('You have unsaved changes. Leave anyway?');  
  };
```

Component implements:

```
@Component({ /* ... */ })  
export class EditUser implements HasUnsavedChanges {  
  formDirty = false;  
  hasUnsavedChanges(): boolean { return this.formDirty; }  
}
```

Route:

```
{  
  path: 'users/:id/edit',  
  component: EditUser,  
  canDeactivate: [unsavedChangesGuard]  
}
```

`confirm()` is browser dialog. In production: modal returning `Observable<boolean>` .

5. `canMatch` — Smarter Than `canActivate`

`canMatch` runs **before** matching. `canActivate` runs after.

- `canMatch` false router skips, continues looking
- `canActivate` false matched but blocked

```
const isAdmin: CanMatchFn = () => inject(AuthService).isAdmin();  
  
export const routes: Routes = [  
  { path: 'dashboard', component: AdminDashboard, canMatch: [isAdmin] },  
  { path: 'dashboard', component: UserDashboard } // fallback if non-admin  
];
```

Use `canMatch` for:

- Lazy-loading on condition (don't load admin chunk for non-admins)
- A/B testing, feature flags
- Role-based layouts

Junior level: know it exists, know it's "the version that lets URL fall through."

6. resolve — Pre-fetch Data

```
export const userResolver: ResolveFn<User> = (route) => {  
  const id = Number(route.paramMap.get('id'));  
  return inject(UserService).getById(id);  
};
```

```
{  
  path: 'users/:id',  
  component: UserDetail,  
  resolve: { user: userResolver }  
}
```

Component:

```
@Component({ /* ... */ })  
export class UserDetail {  
  private route = inject(ActivatedRoute);  
  user = this.route.snapshot.data['user'] as User;  
}
```

Trade-off: user waits longer (no UI until data), but no loading flash.

Often you'd rather fetch in the component with `toSignal()` and show a loading state.
Resolvers for cases where loading flash is jarring.

7. Stacking Guards

```
{  
  path: 'admin',  
  component: AdminPanel,  
  canActivate: [authGuard, adminGuard]  
}
```

Order matters — `authGuard` first. If it fails, `adminGuard` not called.

For child routes, `canActivateChild` :

```
{
  path: 'admin',
  canActivateChild: [AuthGuard, adminGuard],
  children: [
    { path: 'users', component: AdminUsers },
    { path: 'posts', component: AdminPosts }
  ]
}
```

8. Old Class-Based Pattern

```
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}

  canActivate(): boolean | UrlTree {
    return this.auth.isLoggedIn() ? true : this.router.createUrlTree(['/login']);
  }
}
```

Wiring: `canActivate: [AuthGuard]` . Pre-Angular 15 pattern. New code uses functional.

Migrate with `mapToCanActivate([AuthGuard])` or rewrite as functional.

Common Mistakes

1. **Forgetting `take(1)` on long-lived Observables** — hangs navigation.
2. **Using `router.navigate()` inside guard instead of returning `UrlTree`** — fragile.
3. **Business logic in guards** — they should check; services compute.
4. **Forgetting async error handling** — `catchError` important.
5. **Using `canActivate` where `canMatch` is right** — lazy routes still downloaded.
6. **Returning `Promise` the route doesn't await** — usually works in modern Angular.

Interview Soundbites

Q: What are route guards?

"Functions called by router before allowing navigation. Return `true`, `false`, or `UrlTree`. Async via `Observable/Promise`. `canActivate` for auth checks, `canDeactivate` for unsaved-changes, `canMatch` for conditional routing, `resolve` for pre-fetching."

Q: Auth guard implementation?

"Functional guard injects auth service and router. If logged in, return `true`. Otherwise return `router.createUrlTree(['/login'])`. For async, return `Observable` piped with `take(1)`."

Q: `canActivate` vs `canMatch` ?

"`canActivate` runs after route matched — blocks but route owns URL. `canMatch` runs before matching — if false, router skips this route, continues looking. Use `canMatch` for role-based alternative routes or feature-flagged routes."

Q: Unsaved changes warning?

"`canDeactivate` guard receiving component instance. Component implements interface with `hasUnsavedChanges()` method. Guard checks, shows confirmation if dirty, returns based on user choice."

Mental Model

Five sentences:

1. **Guard** = function returning `true`, `false`, or `UrlTree`. Router uses it to decide.
 2. `canActivate` = "can enter?", `canDeactivate` = "can leave?", `canMatch` = "should router consider this route at all?"
 3. **Functional guards** are modern — functions using `inject()`.
 4. **For redirects**, return `router.createUrlTree([...])` — declarative.
 5. **Async** returns `Observable` — `take(1)` so it completes.
-

Topic 21 — Lazy Loading

By default, every component is in the initial bundle. **Lazy loading** splits the app — chunks load only when the user navigates to that route. Smaller initial bundle, faster first paint.

1. loadComponent — Single Component

```
export const routes: Routes = [
  { path: '', component: Home },

  {
    path: 'admin',
    loadComponent: () =>
      import('./admin/admin').then(m => m.Admin)
  }
];
```

The dynamic `import()` tells the build tool: separate bundle. The chunk downloads when user visits `/admin`.

```
// admin/admin.ts
@Component({ /* ... */ })
export class Admin {}
```

2. loadChildren — Whole Section

For a section with multiple routes, bundle together:

```
export const routes: Routes = [
  { path: '', component: Home },
  {
    path: 'admin',
    loadChildren: () =>
      import('./admin/admin.routes').then(m => m.ADMIN_ROUTES)
  }
];
```

```
// admin/admin.routes.ts
export const ADMIN_ROUTES: Routes = [
  { path: '', component: AdminUsers },
  { path: 'products', component: AdminProducts },
```

```
{ path: 'settings', component: AdminSettings }  
];
```

All three pages bundle into one chunk. Navigating between admin pages = instant after first load.

Modern equivalent of old `loadChildren: () => import('./admin.module').then(m => m.AdminModule)` .

3. When to Use Which

Pattern	Use when
<code>loadComponent</code>	Single isolated route (settings page)
<code>loadChildren</code>	Feature section with multiple routes
Eager (neither)	Critical path: home, login, navbar

Eager what users see immediately. Lazy everything else.

4. Lazy + Layout Components

```
// admin/admin.routes.ts  
export const ADMIN_ROUTES: Routes = [  
  {  
    path: "",  
    component: AdminLayout,  
    children: [  
      { path: "", component: AdminUsers },  
      { path: 'products', component: AdminProducts }  
    ]  
  }  
];
```

```
<!-- admin-layout.html -->  
<aside>  
  <a routerLink="">Users</a>  
  <a routerLink="products">Products</a>
```

```
</aside>
<main>
  <router-outlet></router-outlet>
</main>
```

Layout + children all in the lazy chunk.

5. Lazy + Guards

`canActivate` and `canMatch` work — but choose carefully:

```
{
  path: 'admin',
  loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES),
  canActivate: [authGuard] // chunk IS downloaded before guard runs
}
```

vs

```
{
  path: 'admin',
  loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES),
  canMatch: [authGuard] // chunk NOT downloaded if guard fails
}
```

For auth/role checks on lazy routes, **prefer `canMatch`** — don't ship bytes the user can't use.

6. Preloading — Best of Both Worlds

After initial load, download lazy chunks in background.

```
import { provideRouter, withPreloading, PreloadAllModules } from '@angular/router';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes, withPreloading(PreloadAllModules))
  ]
}
```

```
]
};
```

Custom strategy:

```
@Injectable({ providedIn: 'root' })
export class SelectivePreloadStrategy implements PreloadingStrategy {
  preload(route: Route, load: () => Observable<unknown>): Observable<unknown> {
    return route.data?.['preload'] ? load() : of(null);
  }
}
```

```
{ path: 'admin', loadChildren: () => ..., data: { preload: true } }
```

7. @defer — Template-Level Lazy Loading (Angular 17+)

Lazy load parts of templates:

```
<h1>Article Title</h1>
<p>Main content...</p>

@defer (on viewport) {
  <app-comments [articleId]="id"></app-comments>
} @placeholder {
  <p>Comments will appear when you scroll down.</p>
} @loading {
  <p>Loading comments...</p>
}
```

Triggers:

- on viewport — when scrolled into view
- on idle — when browser idle
- on interaction — on hover/click placeholder
- on hover
- on timer(2s) — after delay
- when condition() — custom condition (signal works)

Junior level: know the syntax and value.

8. Verifying It Works

Production build output

```
ng build
```

Look for:

Lazy Chunk Files	Names	Size
chunk-XXXX.js	admin-routes	38.4 kB

If admin appears under “Lazy Chunk Files” — working.

Network tab

- Open app at `/` — note loaded JS
- Navigate to `/admin` — should see NEW JS download
- Navigate back, then forward — no new download (cached)

9. Realistic Project Structure

src/app/	
app.ts	eager
app.config.ts	
app.routes.ts	
core/	eager, app-wide
shared/	eager, used everywhere
home/	eager (initial page)
auth/	lazy
admin/	lazy
account/	lazy

```
export const routes: Routes = [  
  { path: "", component: Home },  
  {
```

```

    path: 'auth',
    loadChildren: () => import('./auth/auth.routes').then(m => m.AUTH_ROUTES)
  },
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES),
    canActivate: [authGuard, adminGuard]
  },
  { path: '**', component: NotFound }
];

```

Common Mistakes

1. **Lazy-loading home page** — user hits it first; lazy chunk loads immediately anyway.
2. **Lazy-loading something used in initial route** — chunk loads immediately.
3. **Cross-imports between lazy and eager** — pulls things into main bundle.
4. **loadComponent for multi-route section** — multiple chunks for related stuff.
5. **canActivate instead of canActivate** for protected lazy routes — chunk still downloads.
6. **Default export confusion** — `m.default` vs `m.SomeName` . Pick named exports.
7. **Routes file using app.config.ts style** — lazy chunks export routes arrays.
8. **Not verifying** — check build output.

Interview Soundbites

Q: What is lazy loading?

"Splitting app into chunks so router downloads code only when needed. Main way to keep initial bundle small. Use `loadComponent` for single component or `loadChildren` pointing at routes array for a section."

Q: How set up?

" `loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES)` . The dynamic import tells build tool to emit a separate chunk that downloads on navigation."

Q: loadComponent vs loadChildren ?

" `loadComponent` for one standalone component on one route. `loadChildren` for routes array — whole section that shares a chunk."

Q: How make navigation feel fast?

“Preloading — after initial bundle, download lazy chunks in background. Simplest: `withPreloading(PreloadAllModules)` in `provideRouter`.”

Q: Lazy routes + guards?

“Prefer `canMatch` over `canActivate` for auth/role gates on lazy routes. `canActivate` runs AFTER matching — chunk already downloaded. `canMatch` runs BEFORE — router skips, chunk never downloaded.”

Q: What's `@defer` ?

“Template-level lazy loading from Angular 17. Wrap component in `@defer` block; Angular splits it into separate chunk that loads on a trigger — `on viewport` , `on interaction` , `on idle` . For deferring below-the-fold, comments, charts, editors.”

Mental Model

Five sentences:

1. `loadComponent` / `loadChildren` use dynamic imports build tool emits separate bundles.
 2. Eager what's on the critical path; lazy everything else.
 3. For protected lazy routes, `canMatch` over `canActivate` — prevent chunk download for unauthorized.
 4. Preloading downloads lazy chunks in background for instant later navigations.
 5. `@defer` brings lazy loading into templates — content-level splitting.
-

Topic 22 — Template-Driven Forms

Angular has two form systems. **Template-driven forms (TDF)** keep logic in the template via `ngModel` and validators-as-attributes. Reactive forms (next topic) put logic in the class.

TDF is rarely the right choice in production, but you need to know it for interviews and legacy code.

1. Setup

Needs `FormsModule` :

```

import { FormsModule } from '@angular/forms';

@Component({
  imports: [FormsModule],
  template: `
    <input [(ngModel)]="email" name="email" />
    <input [(ngModel)]="password" name="password" type="password" />
  `,
})
export class Login {
  email = '';
  password = '';
}

```

TDF = two-way binding (topic 4) + validation directives.

2. The Form Wrapper — **ngForm**

```

<form #loginForm="ngForm" (ngSubmit)="onSubmit(loginForm)">
  <input
    name="email"
    [(ngModel)]="email"
    required
    email />

  <input
    name="password"
    [(ngModel)]="password"
    required
    minlength="6"
    type="password" />

  <button type="submit" [disabled]="!loginForm.valid">Log in</button>
</form>

```

Three key things:

1. **#loginForm="ngForm"** — template ref form object
2. **name="email"** — REQUIRED for each input. Becomes key in form value

3. **(ngSubmit)** — fires on submit, handles preventDefault

loginForm gives you:

```
loginForm.value    // { email: '...', password: '...' }  
loginForm.valid  
loginForm.invalid  
loginForm.dirty  
loginForm.touched  
loginForm.controls
```

3. Built-in Validators

HTML attributes:

```
<input  
  name="username"  
  [(ngModel)]="username"  
  required  
  minlength="3"  
  maxlength="20"  
  pattern="[a-zA-Z0-9]+"  
>
```

```
<input  
  name="email"  
  [(ngModel)]="email"  
  required  
  email  
>
```

```
<input  
  name="age"  
  [(ngModel)]="age"  
  type="number"  
  min="18"  
  max="100"  
>
```

Validator	Checks
required	Not empty
minlength="n"	String $\geq$ n
maxlength="n"	String $\leq$ n
min="n"	Number $\geq$ n
max="n"	Number $\leq$ n
pattern="regex"	Regex match
email	Email format

4. Showing Validation Errors

```

<form #form="ngForm" (ngSubmit)="onSubmit(form)">
  <label>
    Email:
    <input
      name="email"
      [(ngModel)]="email"
      #emailCtrl="ngModel"
      required
      email />
    </label>

    @if (emailCtrl.invalid && emailCtrl.touched) {
      <small class="error">
        @if (emailCtrl.errors?.['required']) { Required. }
        @if (emailCtrl.errors?.['email']) { Invalid email. }
      </small>
    }

    <button type="submit" [disabled]="form.invalid">Submit</button>
  </form>

```

#emailCtrl="ngModel" individual control's state.

UX rule: Show errors after `touched && invalid` , not `invalid` alone. Otherwise every field is red on load.

5. Control State

Property	Means	Opposite
<code>pristine</code>	Never changed	<code>dirty</code>
<code>untouched</code>	Never blurred	<code>touched</code>

6. Auto-Applied CSS Classes

Angular adds:

```
input.ng-valid { border-color: green; }  
input.ng-invalid { border-color: red; }  
input.ng-pristine, input.ng-dirty, input.ng-touched, input.ng-untouched
```

Style errors with pure CSS:

```
input.ng-invalid.ng-touched { border: 1px solid red; }
```

7. Complete Example

```
import { Component } from '@angular/core';  
import { FormsModule, NgForm } from '@angular/forms';  
  
@Component({  
  imports: [FormsModule],  
  template: `  
    <form #form="ngForm" (ngSubmit)="onSubmit(form)" novalidate>  
      <div>  
        <label>Email</label>  
        <input  
          name="email"  
          [(ngModel)]="model.email"  
          #emailCtrl="ngModel"  
        >
```

```

        required
      email />
      @if (emailCtrl.invalid && emailCtrl.touched) {
        <small class="error">
          @if (emailCtrl.errors?.['required']) { Required. }
          @if (emailCtrl.errors?.['email']) { Invalid email. }
        </small>
      }
    </div>

    <div>
      <label>Password</label>
      <input
        name="password"
        type="password"
        [(ngModel)]="model.password"
        #pwCtrl="ngModel"
        required
        minlength="6" />
      @if (pwCtrl.invalid && pwCtrl.touched) {
        <small class="error">
          @if (pwCtrl.errors?.['required']) { Required. }
          @if (pwCtrl.errors?.['minlength']) { At least 6 characters. }
        </small>
      }
    </div>

    <button type="submit" [disabled]="form.invalid">Log in</button>
  </form>
  `;
  styles: [`
    .error { color: red; font-size: 0.85rem; }
    input.ng-invalid.ng-touched { border-color: red; }
  `]
})
export class Login {
  model = { email: "", password: "" };

  onSubmit(form: NgForm) {
    if (form.invalid) return;
  }
}

```

```
    console.log('Submitting', form.value);
  }
}
```

`novalidate` disables browser's native HTML5 validation popups.

8. When to Use TDF

In practice — rarely. Best for:

- Login / signup
- Simple contact / subscribe forms
- Quick prototypes
- Fixed-structure forms

Don't use for:

- Dynamic forms
- Cross-field validation
- Forms you'll test thoroughly
- Complex computed state

9. TDF vs Reactive

	TDF	Reactive
Logic	Template	Class
Setup	<code>FormsModule</code> , <code>ngModel</code>	<code>ReactiveFormsModule</code> , <code>FormGroup</code> / <code>FormControl</code>
Form model	Implicit	Explicit
Validation	Directives	Validator functions
Dynamic forms	Awkward	Native (<code>FormArray</code>)
Testing	Through DOM	Direct
Type safety	Weak	Strong (since v14)
Production	Rare	Standard

Common Mistakes

1. **Forgetting** `name="..."` — `ngModel` won't register.
2. **Forgetting** `FormsModule` — “Can't bind to 'ngModel'”.
3. **Showing errors immediately** — gate with `touched` .
4. **Plain click handler instead of** `(ngSubmit)` — loses Enter-key submit.
5. **Mixing TDF and reactive** in same form.
6. **Reaching form value from outside** — awkward in TDF.
7. **Treating TDF as “modern”** — it's older and simpler, doesn't scale.

Interview Soundbites

Q: Two types of forms?

“Template-driven and reactive. TDF keeps logic in template via `ngModel` and validators-as-attributes. Reactive puts model in class with `FormGroup` / `FormControl` , validator functions. Reactive preferred for real-world — scales better, dynamic fields, easier testing.”

Q: When use TDF?

“Simple, fixed forms — login, signup, contact. Most teams standardize on reactive even for simple cases.”

Q: What's `ngModel` ?

“Directive for two-way binding `[(ngModel)]='property'` . Inside `<form>` , also registers as form control with parent `NgForm` . Input must have `name` attribute.”

Q: Show validation errors in TDF?

“Template ref `#emailCtrl='ngModel'` individual state. Show messages when `invalid && touched` . Read errors via `emailCtrl.errors?.['required']` .”

Mental Model

Three sentences:

1. **TDF = `[(ngModel)]` + `name` + `<form>` with `#form="ngForm"` . Implicit form model.**
2. **Validation = HTML attributes; errors check `touched && invalid` .**
3. **Simple forms only. For anything bigger, reactive forms.**

, `Validators.email`], `[uniqueEmailValidator(this.http)]`],

`name`: `[', [Validators.required, noWhitespace]]`,

`password`: `[', [Validators.required, Validators.minLength(8)]`,


```
confirmPassword: ['', Validators.required]
}, { validators: passwordsMatch });

onSubmit() {
  if (this.form.invalid) {
    this.form.markAllAsTouched();
    return;
  }
  console.log('Submitting:', this.form.value);
}
}
```

Common Mistakes

1. **Forgetting to return `null`** for valid case — control stays invalid.
2. **Calling validators at wrong arity** — `Validators.minLength` is a factory; `Validators`
3. **Async without debounce** — request per keystroke.
4. **`setValidators` without `updateValueAndValidity`** — won't run.
5. **Cross-field validator on a single control** — must be on FormGroup.
6. **Reading errors before touch** — gate with `touched`.
7. **`form.errors?.['x']` vs `form.controls.x.errors?.['y']`** — group vs control level.
8. **Async error rejecting** — wrap with `catchError` returning `null`.

Interview Soundbites

> **Q: How does Angular validation work?**

> **"Each control has list of validator functions. After every value change, Angular runs th**

> **Q: Cross-field validation (password confirmation)?**

> **"Validator on the FormGroup, not control. Reads both fields from the group, returns**

> **Q: Async validators?**

> **"Functions returning Observable or Promise of `ValidationErrors | null`. Third array s**

> **Q: Show server errors on submit?**

> **"Catch in submit handler. For 422 with field-level errors, iterate and `form.get(field)?.s**

Mental Model

Five sentences:

1. **Validator** = function returning `ValidationErrors | null`. Use `Validators.*` for built-in
2. **Parameterized validators** are factories — outer function takes params, returns inner
3. **Cross-field validators** go on `FormGroup`; access sibling controls via `group.get('x')`.
4. **Async validators** return `Observable` — debounce, `catchError(() => of(null))`, third arg
5. `setErrors` for server-side errors; `updateValueAndValidity()` after `setValidators` to

Topic 25 — Change Detection

The mechanism Angular uses to keep the DOM in sync with your component state. Concept

1. The Problem

```
``typescript
@Component({ template: `<p>Count: {{ count }}</p>` })
export class App {
  count = 0;
  increment() { this.count++; }
}
```

Click button `count++` . **How does Angular know to update the DOM?**

Answer: change detection.

2. Zone.js (Classic)

Angular bundles Zone.js, which **monkey-patches** browser async APIs:

- `setTimeout` / `setInterval`
- `addEventListener`
- `Promise.then`
- `XHR` / `fetch`

Each time one of these fires/resolves, Zone.js notifies Angular. Angular runs change detection — checks every component, updates DOM where state changed.

You don't write code to trigger CD. Click handler runs → CD runs.

3. The Default Strategy

For every CD cycle (every event):

1. Start at root.
2. For each component, compare each template binding to last value.
3. If different, update DOM.
4. Recurse to children.

For a 200-component app, that's a lot of work per click. Usually fine, but problematic with:

- Heavy templates
- Many bindings
- Frequent events (mousemove, scroll)

4. OnPush Strategy

```
import { ChangeDetectionStrategy } from '@angular/core';

@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<p>Count: {{ count }}</p>`
})
```

Component re-checked **only when**:

1. @Input reference changes
2. Event fires inside component
3. Observable bound via async pipe emits
4. Signal read by component changes
5. Manual markForCheck() / detectChanges()

External events that don't touch this component? **Skipped**. Big win at scale.

5. The Reference Trap

```
@Component({
  selector: 'app-list',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `@for (i of items; track i) { <p>{{ i }}</p> }`
})
```

```

export class List {
  @Input() items: string[] = [];
}

@Component({ /* parent */ })
export class Parent {
  items = ['a', 'b'];

  addItem() {
    this.items.push('c'); // same reference, OnPush won't re-check
  }
}

```

Fix — replace, don't mutate:

```

this.items = [...this.items, 'c']; // new reference

```

This is why **immutable updates** and **OnPush** go together.

6. OnPush + async Pipe — Common Combo

```

@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @for (u of users$ | async; track u.id) {
      <p>{{ u.name }}</p>
    }
  `,
})
export class UserList {
  users$ = inject(HttpClient).get<User[]>('/api/users');
}

```

async calls **markForCheck()** on emission. OnPush component still updates. Works perfectly together.

7. OnPush + Signals

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<p>Count: {{ count() }}</p>`
})
export class Counter {
  count = signal(0);
  increment() { this.count.update(c => c + 1); }
}
```

Signal change Angular knows to re-check this component. **No** `markForCheck` needed.

This is why signals + OnPush are the modern recommended pattern.

8. Manual Triggers

```
import { ChangeDetectorRef } from '@angular/core';

private cdr = inject(ChangeDetectorRef);

this.cdr.markForCheck(); // mark for next CD cycle
this.cdr.detectChanges(); // run CD NOW on this component subtree
this.cdr.detach(); // stop CD entirely
this.cdr.reattach(); // resume
```

`markForCheck` is what you'd use 95% of the time when needed. `detectChanges` is for surgical synchronous updates (rare).

9. Running Outside Angular

For tight loops that don't touch UI:

```
import { NgZone } from '@angular/core';
private zone = inject(NgZone);

this.zone.runOutsideAngular(() => {
  setInterval(() => {
    this.analytics.tick(); // doesn't trigger CD
  }, 1000);
});
```

```
}, 100);  
});
```

When you do need a UI update, hop back:

```
this.zone.run(() => this.count.update(c => c + 1));
```

10. Zoneless (Angular 18+)

The future. Drop Zone.js:

```
providers: [provideExperimentalZonelessChangeDetection()]
```

Angular relies on signals, `async` pipe, and explicit `markForCheck()` to know when to re-check. Smaller bundle, more predictable CD, no monkey-patching weirdness.

Junior level: know it exists, signals are how you get there.

11. Performance Diagnostics

Open **Angular DevTools** browser extension Profiler tab Record. See which components ran CD per cycle and how long each took.

If a small interaction causes 200 component checks, you have a CD problem. OnPush + signals is the fix.

Common Mistakes

1. **Mutating arrays/objects on OnPush** — UI doesn't update.
2. **Expecting CD to run on plain property change** — usually does (Zone), but not after OnPush gate.
3. **Heavy expressions in templates** — `{{ expensiveCalc() }}` runs on every CD. Use `computed()` or pipes.
4. **Manual `detectChanges()` in tight loops** — Angular's job; rarely needed.
5. **Async outside Angular without re-entering** — UI doesn't update.

Interview Soundbites

Q: How does change detection work?

"Angular uses Zone.js by default — it patches async APIs. Each time one fires (click, timer, HTTP), Angular runs CD: walks component tree, compares each binding's current value to last, updates DOM where different."

Q: What does OnPush do?

"Change CD strategy from running on every event to only running when: @Input reference changes, event inside component, async pipe emits, signal read by component changes, or markForCheck() called. Much less work, especially in large apps."

Q: Why doesn't my OnPush UI update?

"Probably mutating an object instead of replacing. OnPush checks by reference. this.items.push(x) keeps same reference; OnPush sees no change. Use this.items = [...this.items, x] instead."

Q: Signals and CD?

"Signals tell Angular exactly which components depend on which values. With OnPush + signals, Angular re-checks only affected subtrees. Foundation for zoneless mode — eventually Zone.js will be optional."

Mental Model

Five sentences:

1. **Zone.js** patches async APIs; each fires triggers a CD pass over the tree.
2. **Default strategy = check everyone; OnPush = check only when input ref changes, event fires, async pipe emits, signal changes, or markForCheck() called.**
3. **OnPush requires immutable updates — replace, don't mutate.**
4. **Signals tell Angular precisely what changed fine-grained CD; future is zoneless.**
5. **For non-UI background work, zone.runOutsideAngular(...); hop back with zone.run(...).**

Topic 26 — Content Projection (<ng-content>)

Projection = letting a parent inject HTML into a child's template. Like `children` in React, slots in Vue/Web Components.

1. The Idea

You want a `<app-card>` you can put anything inside:

```
<app-card>
  <h2>Welcome</h2>
  <p>This goes inside the card.</p>
</app-card>
```

The card adds styles, headers, borders. Caller provides content. **Projection.**

2. Single-Slot `<ng-content>`

```
@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <ng-content></ng-content>
    </div>
  `,
  styles: [`.card { border: 1px solid #ccc; padding: 16px; border-radius: 8px; }`],
})
export class Card {}
```

`<ng-content></ng-content>` is the slot. Whatever the parent puts between `<app-card>` and `</app-card>` ends up there.

Use:

```
<app-card>
  <h2>Welcome</h2>
  <p>Hello, world.</p>
</app-card>
```

Output:


```

<app-card>
  <div class="card">
    <h2>Welcome</h2>
    <p>Hello, world.</p>
  </div>
</app-card>

```

3. Multi-Slot — select

For header/body/footer regions:

```

@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <header><ng-content select="[card-header]"></ng-content></header>
      <main> <ng-content></ng-content></main>
      <footer><ng-content select="[card-footer]"></ng-content></footer>
    </div>
  `
})
export class Card {}

```

`select="[card-header]"` is a CSS attribute selector. Matches anything with `card-header` attribute.

```

<app-card>
  <h2 card-header>Article Title</h2>

  <p>Body content here.</p>
  <p>More body content.</p>

  <button card-footer>Save</button>
</app-card>

```

- `<h2>` header slot
- `<button>` footer slot
- Anything not matched default (unselected) slot

Other selector types:

```
<ng-content select="h2"></ng-content>    // by tag
<ng-content select=".highlight"></ng-content> // by class
<ng-content select="app-icon"></ng-content> // by component
```

4. Scope Gotcha

Projected content belongs to **caller's scope**, not the wrapper's:

```
<!-- in parent -->
<app-card>
  <p>{{ title }}</p> <!-- 'title' is from PARENT, not Card -->
</app-card>
```

If `title` doesn't exist on parent, error. Card has no access to its children's variables.

5. ngProjectAs — Override the Match

If a wrapper element doesn't match the selector:

```
<app-card>
  <div ngProjectAs="[card-header]">
    <span>Header in a wrapper</span>
  </div>

  <p>Body content.</p>
</app-card>
```

Even though `<div>` has no `card-header` attribute, `ngProjectAs="[card-header]"` tells Angular to project it as if it did.

Use case: structural directives wrapping content (`@if` -style logic).

6. With Conditional Content

```
<app-card>
  <h2 card-header>Profile</h2>
```

```

@if (loading) {
  <p>Loading...</p>
} @else {
  <p>{{ user.name }}</p>
}
</app-card>

```

The `@if` evaluates in parent scope. The matched chunk gets projected.

7. Advanced — Templated Projection

For projecting templates the wrapper can render with its own data:

```

@Component({
  selector: 'app-list',
  template: `
    @for (item of items; track item.id) {
      <ng-container *ngTemplateOutlet="rowTemplate; context: { $implicit: item }"></ng-c
    }
  `,
})
export class List {
  @Input() items: any[] = [];
  @ContentChild('row', { read: TemplateRef }) rowTemplate!: TemplateRef<any>;
}

```

Used:

```

<app-list [items]="users">
  <ng-template #row let-user>
    <div class="row">
      {{ user.name }} — {{ user.email }}
    </div>
  </ng-template>
</app-list>

```

Render-prop pattern. Wrapper supplies data + structure; caller supplies markup. Junior level: recognize this pattern.

8. `<ng-content>` vs `<ng-container>` vs `<ng-template>`

	What
<code><ng-content></code>	Slot for projected content
<code><ng-container></code>	Invisible wrapper — group without adding DOM
<code><ng-template></code>	Defines a template that doesn't render unless used (<code>*ngIf</code> , <code>*ngTemplateOutlet</code>)

```
<!-- ng-container groups without wrapping div -->
@if (showSection) {
  <ng-container>
    <h3>Title</h3>
    <p>Description</p>
  </ng-container>
}
```

9. Realistic Example — Modal

```
@Component({
  selector: 'app-modal',
  template: `
    @if (isOpen) {
      <div class="modal-backdrop" (click)="close.emit()">
        <div class="modal" (click)="$event.stopPropagation()">
          <header class="modal-header">
            <ng-content select="[modal-title]"></ng-content>
            <button (click)="close.emit()">x</button>
          </header>

          <main class="modal-body">
            <ng-content></ng-content>
          </main>

          <footer class="modal-footer">
```

```

    <ng-content select="[modal-footer]"></ng-content>
  </footer>
</div>
</div>
}
,
})
export class Modal {
  @Input() isOpen = false;
  @Output() close = new EventEmitter<void>();
}

```

```

<app-modal [isOpen]="showConfirm" (close)="showConfirm = false">
  <h2 modal-title>Delete user?</h2>

  <p>This cannot be undone.</p>
  <p>User: <strong>{{ user.name }}</strong></p>

  <div modal-footer>
    <button (click)="showConfirm = false">Cancel</button>
    <button (click)="deleteUser()">Delete</button>
  </div>
</app-modal>

```

Modal owns structure + styling. Caller owns content. **The Card/Modal/Tabs/Dialog pattern.**

Common Mistakes

1. **Putting content outside** `<ng-content>` — silently dropped or rendered without slot.
2. **Multiple unselected** `<ng-content>` — only first gets unselected content.
3. **Confusing** `<ng-content>` **with** `<ng-template>` — they're different.
4. **Scope confusion** — content evaluates in caller's scope, not wrapper's.
5. **Heavy logic in projected content** — runs in caller's CD, not wrapper's.

Interview Soundbites

Q: What's content projection?

"`<ng-content>` lets parent component pass markup into child's template. The slot mechanism — like `children` in React. Caller provides content, wrapper provides layout/styles."

Q: How project multiple slots?

"Multiple `<ng-content>` elements with `select` attribute, each taking a CSS selector. `<ng-content select=\"[card-header]\">` matches elements with that attribute. Default `<ng-content>` (no `select`) catches unmatched."

Q: Scope of projected content?

"Caller's scope. If parent's template has `<app-card>{{ title }}</app-card>`, `title` is on parent, not on Card. Wrapper has no access to caller's data."

Mental Model

Four sentences:

1. `<ng-content>` = slot. Caller's markup gets placed there.
2. For multiple slots, use `select` with CSS selector (attribute, class, tag).
3. Projected content evaluates in caller's scope, not wrapper's.
4. For data-driven slots (parent passes data into a template), use `<ng-template>` + `ngTemplateOutlet`.

Topic 27 — ViewChild and ContentChild

References to template elements/components — when you need to imperatively interact (call a method, read DOM, focus).

1. @ViewChild — Reference Inside Own Template

```
import { Component, ViewChild, ElementRef, AfterViewInit } from '@angular/core';

@Component({
  template: `
    <input #emailInput type="email" />
    <button (click)="focusEmail()">Focus</button>
  `
})
```

```

})
export class Login implements AfterViewInit {
  @ViewChild('emailInput') emailInput!: ElementRef<HTMLInputElement>;

  ngAfterViewInit() {
    this.emailInput.nativeElement.focus();
  }

  focusEmail() {
    this.emailInput.nativeElement.focus();
  }
}

```

- `#emailInput` in template — template reference variable
- `@ViewChild('emailInput')` matches it
- `ElementRef.nativeElement` = raw DOM element

2. Referencing a Component

```

@Component({
  template: `<app-counter #counter></app-counter>`
})
export class Parent implements AfterViewInit {
  @ViewChild('counter') counter!: Counter;

  ngAfterViewInit() {
    this.counter.increment(); // call method on child
  }
}

```

Or by type (no template variable needed):

```

@ViewChild(Counter) counter!: Counter;

```

3. When References Are Available

```
ngOnInit() { this.emailInput.nativeElement.focus(); } // undefined
ngAfterViewInit() { this.emailInput.nativeElement.focus(); } //
```

`@ViewChild` resolves **after** the view is rendered. Use `ngAfterViewInit` .

For static refs (not inside `@if / @for`), available earlier with `{ static: true }` :

```
@ViewChild('emailInput', { static: true }) emailInput!: ElementRef;
// Available in ngOnInit
```

99% of the time use the default (`static: false`) + `ngAfterViewInit` .

4. @ViewChildren — Multiple Refs

```
import { ViewChildren, QueryList } from '@angular/core';

@Component({
  template: `
    @for (item of items; track item.id) {
      <app-product-card #card [product]="item"></app-product-card>
    }
  `,
})
export class ProductList implements AfterViewInit {
  @ViewChildren('card') cards!: QueryList<ProductCard>;

  ngAfterViewInit() {
    console.log('count:', this.cards.length);
    this.cards.forEach(c => c.refresh());

    this.cards.changes.subscribe(() => {
      console.log('list changed:', this.cards.length);
    });
  }
}
```

`QueryList` :

- `.length` , `.toArray()` , `.first` , `.last`

- `.forEach` , `.map` , `.filter`
- `.changes` Observable when contents change

5. `@ContentChild` — Reference Projected Content

```
@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <ng-content></ng-content>
    </div>
  `,
})
export class Card implements AfterContentInit {
  @ContentChild('title', { read: ElementRef })
  title?: ElementRef<HTMLElement>;

  ngAfterContentInit() {
    console.log('Projected title text:', this.title?.nativeElement.textContent);
  }
}
```

```
<app-card>
  <h2 #title>Welcome</h2>
  <p>Body</p>
</app-card>
```

`@ViewChild` = own template. `@ContentChild` = projected content. Available in `ngAfterContentInit`.

Used by component libraries (like Material's `MatTab`) — they project content but read structure.

6. The `read` Option

What's returned depends on `read`:

```

@ViewChild('input') inputRef!: ElementRef;
// Default: depends on element. For DOM elements ElementRef. For components co

@ViewChild('input', { read: ElementRef }) inputRef!: ElementRef;
// Force ElementRef

@ViewChild(Counter, { read: ElementRef }) counterEl!: ElementRef;
// Get DOM wrapper instead of component instance

```

7. Modern API — `ViewChild()` / `contentChild()` Signal-Based

```

import { ViewChild, viewChildren, contentChild } from '@angular/core';

@Component({
  template: `<input #emailInput />`
})
export class Login {
  emailInput = viewChild<ElementRef<HTMLInputElement>>('emailInput');

  focus() {
    this.emailInput()?.nativeElement.focus();
  }
}

```

Differences:

- Returns a **signal** — call it: `this.emailInput()`
- Reactive — re-runs computed/effects on change
- Required version: `ViewChild.required('emailInput')`

Bridging into reactivity nicely:

```

cards = viewChildren(ProductCard);
totalRefreshable = computed(() => this.cards().length);

```

In new code, prefer signal-based. Decorator version remains valid.

8. Realistic Examples

Auto-focus

```
@Component({
  template: `<input #focusMe />`
})
export class Field implements AfterViewInit {
  @ViewChild('focusMe') input!: ElementRef<HTMLInputElement>;
  ngAfterViewInit() { this.input.nativeElement.focus(); }
}
```

Scroll to error

```
@ViewChildren('errorField', { read: ElementRef })
errors!: QueryList<ElementRef>;

onSubmit() {
  if (this.form.invalid) {
    this.form.markAllAsTouched();
    setTimeout(() => {
      this.errors.first?.nativeElement.scrollToView({ behavior: 'smooth', block: 'center' });
    });
  }
}
```

Calling child method

```
@Component({
  template: `
    <app-video-player #player></app-video-player>
    <button (click)="player.play()">Play</button>
    <button (click)="player.pause()">Pause</button>
  `
})
export class App {}
```

Often simplest — no `@ViewChild` declaration, use template ref directly.

Reading projected list (tabs)

```
@Component({
  selector: 'app-tabs',
  template: `
    <div class="tab-list">
      @for (t of tabs; track t; let i = $index) {
        <button (click)="active = i">{{ t.label }}</button>
      }
    </div>
    <ng-content></ng-content>
  `,
})
export class Tabs implements AfterContentInit {
  @ContentChildren(Tab) tabs!: QueryList<Tab>;
  active = 0;
  ngAfterContentInit() { /* set up tabs */ }
}
```

9. When NOT to Use

Don't use `ViewChild` to:

1. Pass data between parent and child `@Input / @Output`
2. Share state across components `service`
3. Manipulate child UI broadly `child should own it`

Use when:

1. Calling imperative method (`focus()` , `play()` , `scrollIntoView()`)
2. Reading DOM dimensions
3. Integrating non-Angular library on element
4. Querying projected content for layout (component lib pattern)

Common Mistakes

1. **Using in `ngOnInit`** — undefined (use `ngAfterViewInit`).
2. **Forgetting `!` or `?`** — TS strict mode complains.

3. **@ViewChild** for own template — silently undefined (use **@ViewChild**).
4. **Tightly coupling parent to child internals** — use Inputs/Outputs.
5. **Forgetting { static: true } for ngOnInit** access — only if guaranteed there.

Interview Soundbites

Q: What's @ViewChild ?

*"Decorator to reference an element or child component in own template. Used for imperative interactions — focus, call method, read DOM. Reference available in **ngAfterViewInit** . Reference by template variable **#myRef** or component class."*

Q: @ViewChild vs @ContentChild ?

*"@ViewChild queries own template. @ContentChild queries content projected through **<ng-content>** . Resolution timing differs: **ngAfterViewInit** vs **ngAfterContentInit** ."*

Q: When NOT to use @ViewChild ?

*"For data flow between parent and child. Use **@Input** / **@Output** . ViewChild for imperative needs — focus, DOM read, calling library methods."*

Q: Modern signal API?

*"viewChild() , viewChildren() , contentChild() from @angular/core . Return signals — read with **()** . Reactive, composes with **computed** / **effect** . Preferred in new code."*

Mental Model

Four sentences:

1. **@ViewChild** for own template; **@ContentChild** for projected content.
2. Use **ngAfterViewInit** / **ngAfterContentInit** , not **ngOnInit** .
3. For lists, **@ViewChildren** / **@ContentChildren** return **QueryList** with **.changes** **Observable**.
4. Modern: **viewChild()** / **contentChild()** return signals — preferred in new code.

Topic 28 — Custom Directives

Topic 6 covered the concept. Now: real patterns you'll actually build and see in production codebases.

1. The Basics Recap

```
@Directive({
  selector: '[appHighlight]',
  host: {
    '(mouseenter)': 'onEnter()',
    '(mouseleave)': 'onLeave()'
  }
})
export class Highlight {
  @Input() appHighlight = 'yellow';
  private el = inject(ElementRef);

  onEnter() { this.el.nativeElement.style.backgroundColor = this.appHighlight; }
  onLeave() { this.el.nativeElement.style.backgroundColor = ''; }
}
```

- `selector: '[appHighlight]'` — attribute selector
- `ElementRef` — DOM element
- `host` — listeners and bindings on host element
- `@Input()` — accepts value

2. `Renderer2` for DOM Manipulation

Direct `el.nativeElement.style.xxx` works but ties to browser. `Renderer2` is platform-safe (works in SSR, web workers):

```
import { Renderer2 } from '@angular/core';

@Directive({ selector: '[appHighlight]' })
export class Highlight {
  private el = inject(ElementRef);
  private renderer = inject(Renderer2);

  @Input() appHighlight = 'yellow';
```

```

@HostListener('mouseenter') onEnter() {
  this.renderer.setStyle(this.el.nativeElement, 'background-color', this.appHighlight);
}

@HostListener('mouseleave') onLeave() {
  this.renderer.removeStyle(this.el.nativeElement, 'background-color');
}
}

```

Renderer2 methods: `setStyle` , `removeStyle` , `addClass` , `removeClass` , `setAttribute` , `removeAttribute` , `appendChild` , `removeChild` .

For projects that might SSR: prefer `Renderer2` . Pure SPA: direct access is fine.

3. @HostListener and @HostBinding

Alternative to the `host` object:

```

import { HostListener, HostBinding } from '@angular/core';

@Directive({ selector: '[appHighlight]' })
export class Highlight {
  @Input() appHighlight = 'yellow';

  @HostBinding('style.backgroundColor') backgroundColor: string | null = null;

  @HostListener('mouseenter') onEnter() { this.backgroundColor = this.appHighlight; }
  @HostListener('mouseleave') onLeave() { this.backgroundColor = null; }
}

```

`@HostBinding('style.backgroundColor')` binds property to element's style.
`@HostListener('mouseenter')` listens.

The `host` object form is newer and preferred. Both common.

4. host Bindings Object

```

@Directive({
  selector: '[appHighlight]',

```

```

host: {
  '[style.background-color]': 'backgroundColor', // property
  '[class.is-highlighted]': '!!backgroundColor',
  '[attr.data-highlight]': 'backgroundColor',
  '(mouseenter)': 'onEnter()',           // event
  '(mouseleave)': 'onLeave()'
}
})

```

5. Pattern 1 — Tooltip Directive

```

@Directive({
  selector: '[appTooltip]',
  host: {
    '(mouseenter)': 'show()',
    '(mouseleave)': 'hide()'
  }
})
export class Tooltip {
  @Input('appTooltip') text = '';
  private el = inject(ElementRef);
  private renderer = inject(Renderer2);
  private tooltipEl?: HTMLElement;

  show() {
    if (!this.text) return;
    this.tooltipEl = this.renderer.createElement('div');
    this.renderer.addClass(this.tooltipEl, 'tooltip');
    this.tooltipEl!.textContent = this.text;

    const rect = this.el.nativeElement.getBoundingClientRect();
    this.renderer.setStyle(this.tooltipEl, 'position', 'fixed');
    this.renderer.setStyle(this.tooltipEl, 'top', `${rect.top - 30}px`);
    this.renderer.setStyle(this.tooltipEl, 'left', `${rect.left}px`);

    this.renderer.appendChild(document.body, this.tooltipEl);
  }

  hide() {

```



```

    if (this.tooltipEl) {
      this.renderer.removeChild(document.body, this.tooltipEl);
      this.tooltipEl = undefined;
    }
  }
}

```

```
<button [appTooltip]="Saves your draft">Save</button>
```

`@Input('appTooltip')` — `aliasName` matching selector. Pattern for “the value of this directive”.

6. Pattern 2 — Click Outside

Useful for closing dropdowns/modals when user clicks elsewhere:

```

@Directive({
  selector: '[appClickOutside]',
  host: {
    '(document:click)': 'onDocClick($event)'
  }
})
export class ClickOutside {
  @Output() appClickOutside = new EventEmitter<void>();
  private el = inject(ElementRef);

  onDocClick(event: MouseEvent) {
    if (!this.el.nativeElement.contains(event.target)) {
      this.appClickOutside.emit();
    }
  }
}

```

```

<div class="dropdown" (appClickOutside)="open = false">
  ...
</div>

```

The directive listens on `document`, not the element. Standard pattern.

7. Pattern 3 — Debounced Click

Prevents double-submit:

```
@Directive({
  selector: '[appDebouncedClick]',
  host: { '(click)': 'subject$.next()' }
})
export class DebouncedClick implements OnInit, OnDestroy {
  @Input() debounceMs = 500;
  @Output() debouncedClick = new EventEmitter<void>();

  subject$ = new Subject<void>();
  private sub?: Subscription;

  ngOnInit() {
    this.sub = this.subject$.pipe(
      debounceTime(this.debounceMs)
    ).subscribe(() => this.debouncedClick.emit());
  }

  ngOnDestroy() { this.sub?.unsubscribe(); }
}
```

```
<button appDebouncedClick (debouncedClick)="save()">Save</button>
```

8. Pattern 4 — Form Field Error Styling

Auto-adds `is-invalid` class when form control invalid + touched:

```
@Directive({
  selector: '[formControlName], [formControl]',
  host: {
    '[class.is-invalid]': 'isInvalid()'
  }
})
export class ErrorState {
  private ngControl = inject(NgControl, { optional: true });
```

```
isInvalid(): boolean {  
  return !(this.ngControl?.invalid && this.ngControl?.touched);  
}  
}
```

Add to a module automatic for every form control. Avoids littering `[class.is-invalid]` everywhere.

9. Directives with Generic Types

Type-safe template ref:

```
@Directive({ selector: '[appList]' })  
export class List<T> {  
  @Input() appListOf!: T[];  
}
```

Used like:

```
<ng-template [appListOf]="users" let-item="$implicit">  
  {{ item.name }}  
</ng-template>
```

Advanced. Junior level: know structural directives can be type-safe.

10. When to Use a Directive

Use directive when	Don't use directive when
Add behavior to existing elements	Need a template — use component
Reusable across element types	One-off logic
DOM manipulation pattern	Pure data transformation — use pipe
Wrap a 3rd-party library on element	Cross-component state — use service

Common Mistakes

1. **Wrong selector syntax** — `selector: 'appName'` matches `<appName>` , not attribute. Use `[appName]` .
2. **Direct DOM access in SSR** — `el.nativeElement.x` breaks. Use `Renderer2` .
3. **Forgetting cleanup** — directive subscriptions leak too.
4. **Mutating Input** — should use `@HostBinding` or service.
5. **Forgetting (document:click) cleanup** — Angular's host binding handles it.

Interview Soundbites

Q: What's a directive?

"Class that adds behavior to an existing DOM element. Attribute selectors like `[appHighlight]` . Inject `ElementRef` for DOM access, `Renderer2` for platform-safe manipulation. `host` config for events and bindings, or `@HostListener` / `@HostBinding` ."

Q: Directive vs component?

"Component is a directive with a template. Use directive when adding behavior to existing element (highlight, autofocus, tooltip, click-outside). Use component for self-contained UI piece with own markup."

Q: `Renderer2` ?

"Angular's DOM abstraction — works in SSR, web workers, anywhere not browser. `setStyle` , `addClass` , `createElement` . Prefer over `nativeElement.x` if app might SSR."

Q: Common custom directives?

"Tooltip, click-outside, autofocus, debounced click, infinite scroll, drag-and-drop, copy-to-clipboard, intersection observer. Anything reusable that needs DOM-level access."

Mental Model

Four sentences:

1. **Directive = behavior on existing element.** `[bracket]` selectors, `ElementRef` / `Renderer2` for DOM.
 2. `host` config (or `@HostListener` / `@HostBinding`) wires events and bindings.
 3. Use `Renderer2` over direct DOM if SSR matters.
 4. **Common patterns:** tooltip, click-outside, debounced events, auto-focus, copy-to-clipboard.
-

Topic 29 — HTTP Interceptors

A function that runs before every HTTP request goes out. Use for Authorization headers, logging, error handling, retries, loaders — anything cross-cutting on HTTP.

1. The Idea

Without interceptors:

```
this.http.get('/api/users', { headers: { Authorization: `Bearer ${token}` } });  
this.http.post('/api/orders', body, { headers: { Authorization: `Bearer ${token}` } });  
// ... repeat in 50 places
```

With interceptor: add header once, applies to every request.

2. Functional Interceptor

```
// interceptors/auth.interceptor.ts  
import { HttpInterceptorFn } from '@angular/common/http';  
import { inject } from '@angular/core';  
import { AuthService } from '../services/auth.service';  
  
export const authInterceptor: HttpInterceptorFn = (req, next) => {  
  const auth = inject(AuthService);  
  const token = auth.token();  
  
  if (!token) return next(req);  
  
  const authReq = req.clone({  
    setHeaders: { Authorization: `Bearer ${token}` }  
  });  
  
  return next(authReq);  
};
```

Register:

```
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { authInterceptor } from '../interceptors/auth.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(withInterceptors([authInterceptor]))
  ]
};
```

3. The Pattern

Every interceptor:

1. Takes `req` (request) and `next` (handler)
2. May modify request (or not)
3. Returns `next(req)` — continues the chain
4. Can also tap/transform the response Observable

`HttpRequest` is **immutable** — must clone:

```
const newReq = req.clone({ setHeaders: { 'X-Foo': 'bar' } });
```

4. Multiple Interceptors

```
provideHttpClient(withInterceptors([authInterceptor, loggingInterceptor, errorIntercept
```

Run in order for request, reverse for response. Like middleware.

5. Pattern 1 — Auth Header

Already shown. Real version with skip list:

```
const SKIP_AUTH_URLS = ['/api/login', '/api/signup'];

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  if (SKIP_AUTH_URLS.some(url => req.url.includes(url))) {
    return next(req);
  }
```

```

}

const token = inject(AuthService).token();
if (!token) return next(req);

return next(req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }));
};

```

6. Pattern 2 — Error Handling

```

import { catchError, throwError } from 'rxjs';
import { HttpResponse } from '@angular/common/http';

export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router);
  const toast = inject(ToastService);

  return next(req).pipe(
    catchError((err: HttpResponse) => {
      if (err.status === 401) {
        router.navigate(['/login']);
      } else if (err.status === 403) {
        toast.error('You don\'t have permission for this action.');
```

```

      } else if (err.status >= 500) {
        toast.error('Server error. Try again later.');
```

```

      }
      return throwError(() => err);
    })
  );
};

```

Centralized 401/403/500 handling. Components don't repeat.

7. Pattern 3 — Loading Indicator

```

import { finalize } from 'rxjs';

export const loadingInterceptor: HttpInterceptorFn = (req, next) => {

```

```
const loader = inject(LoaderService);
loader.show();
return next(req).pipe(
  finalize(() => loader.hide())
);
};
```

Service with counter:

```
@Injectable({ providedIn: 'root' })
export class LoaderService {
  private count = 0;
  isLoading = signal(false);

  show() {
    this.count++;
    this.isLoading.set(true);
  }

  hide() {
    this.count = Math.max(0, this.count - 1);
    if (this.count === 0) this.isLoading.set(false);
  }
}
```

Global spinner reads `loader.isLoading()` .

8. Pattern 4 — Logging

```
export const loggingInterceptor: HttpInterceptorFn = (req, next) => {
  const start = Date.now();
  console.log(` ${req.method} ${req.url}`);

  return next(req).pipe(
    tap({
      next: () => console.log(` ${req.method} ${req.url} (${Date.now() - start}ms)`),
      error: e => console.log(` ${req.method} ${req.url} ${e.status} (${Date.now() - start}ms)`),
    })
  );
}
```



```
);  
};
```

9. Pattern 5 — Retry with Backoff

```
import { retry, timer } from 'rxjs';  
  
export const retryInterceptor: HttpInterceptorFn = (req, next) => {  
  if (req.method !== 'GET') return next(req); // Don't retry mutations  
  
  return next(req).pipe(  
    retry({  
      count: 3,  
      delay: (err, attempt) => {  
        if (err.status >= 500) return timer(attempt * 1000);  
        throw err; // Don't retry 4xx  
      }  
    })  
  );  
};
```

Only retry GET, only on 5xx, exponential backoff.

10. Pattern 6 — Refresh Token (Hard)

```
import { BehaviorSubject, filter, switchMap, take } from 'rxjs';  
  
let isRefreshing = false;  
const refreshToken$ = new BehaviorSubject<string | null>(null);  
  
export const refreshInterceptor: HttpInterceptorFn = (req, next) => {  
  const auth = inject(AuthService);  
  
  return next(req).pipe(  
    catchError((err: HttpResponse) => {  
      if (err.status !== 401) return throwError(() => err);  
  
      if (!isRefreshing) {
```

```

isRefreshing = true;
refreshToken$.next(null);

return auth.refresh().pipe(
  switchMap(token => {
    isRefreshing = false;
    refreshToken$.next(token);
    return next(req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }));
  })
);
}

return refreshToken$.pipe(
  filter(t => t !== null),
  take(1),
  switchMap(token =>
    next(req.clone({ setHeaders: { Authorization: `Bearer ${token!}` } }))
  )
);
});
};

```

Queues concurrent 401s until refresh completes. Junior level: know the *idea*, don't worry about writing from scratch.

11. Class-Based (Legacy)

```

@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  constructor(private auth: AuthService) {}

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const token = this.auth.token();
    if (!token) return next.handle(req);

    return next.handle(req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }));
  }
}

```

Old pre-v15 way. Recognize for legacy code. Register with `HTTP_INTERCEPTORS` . New code uses functional.

Common Mistakes

1. **Mutating request directly** — must `clone()` .
2. **Forgetting `next(req)`** — request never sent.
3. **Putting interceptor before `provideHttpClient()`** — order matters in `withInterceptors` .
4. **Synchronous logic where async needed** — interceptors return Observable; respect chain.
5. **Heavy work outside Observable** — runs once per pipe creation, may not behave as expected.
6. **Refresh token without queueing** — multiple parallel requests each trigger refresh.
7. **Retry on 4xx** — usually wrong; don't retry validation/auth errors.

Interview Soundbites

Q: What's an HTTP interceptor?

"Function running before every HTTP request leaves and around the response. Modify request (add auth header), handle response (errors, logging, retry). Registered via `withInterceptors([authInterceptor])` in `provideHttpClient()` ."

Q: How add auth header globally?

"Interceptor reading token from auth service, cloning request with `setHeaders: { Authorization: ... }` , calling `next(req)` . Registered once. Every request gets header automatically."

Q: Why clone the request?

" `HttpRequest` is immutable. `req.clone({ setHeaders: {...} })` returns new request with modifications. Direct mutation throws or silently fails."

Q: Handle 401 globally?

"Interceptor with `catchError` . On 401, navigate to login (and clear token). For refresh-token flows, call refresh endpoint and replay request — gets complex with concurrent 401s; queue via `BehaviorSubject` ."

Mental Model

Five sentences:

1. **Interceptor** = function (req, next) => Observable<HttpEvent> modifying request and/or response.
 2. **Immutable** `HttpRequest` : clone, don't mutate.
 3. **Register** via `provideHttpClient(withInterceptors([...]))` . Multiple = pipeline.
 4. **Common patterns**: auth header, error handler, loader, logger, retry, refresh-token.
 5. Use `tap` / `catchError` / `finalize` to interact with response Observable.
-

Topic 30 — Testing Basics

Junior level: write a few unit tests, understand patterns, talk about them in interviews.
Tests in real teams are mostly maintenance; you'll learn the project's style on day one.

1. The Tools

`ng new` includes:

- **Jasmine** — testing framework (`describe` , `it` , `expect`)
- **Karma** — test runner (newer projects use Jest instead — same patterns)
- **TestBed** — Angular utility for setting up components in tests

Run with: `ng test`

2. Anatomy of a Test

```
import { TestBed } from '@angular/core/testing';
import { Counter } from './counter';

describe('Counter', () => {
  let component: Counter;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [Counter]
    }).compileComponents();
```

```

const fixture = TestBed.createComponent(Counter);
component = fixture.componentInstance;
});

it('should start at 0', () => {
  expect(component.count).toBe(0);
});

it('increments by 1', () => {
  component.increment();
  expect(component.count).toBe(1);
});
});

```

- `describe(name, fn)` — group
- `beforeEach` — set up per test
- `it(name, fn)` — single test
- `expect(value).toBe(expected)` — assertion

Standalone components `imports` . Old NgModule style `declarations` .

3. Testing Template Output

```

import { ComponentFixture, TestBed } from '@angular/core/testing';
import { By } from '@angular/platform-browser';

describe('Counter template', () => {
  let fixture: ComponentFixture<Counter>;

  beforeEach(() => {
    fixture = TestBed.createComponent(Counter);
    fixture.detectChanges();
  });

  it('renders initial count', () => {
    const el = fixture.nativeElement as HTMLElement;
    expect(el.textContent).toContain('Count: 0');
  });

  it('increments on click', () => {

```

```

const button = fixture.debugElement.query(By.css('button')).nativeElement;
button.click();
fixture.detectChanges();

expect(fixture.nativeElement.textContent).toContain('Count: 1');
});
});

```

- `fixture.detectChanges()` — runs change detection. **Required after any change.**
- `nativeElement` — raw DOM
- `debugElement.query(By.css(...))` — Angular-aware querying

4. Testing Inputs and Outputs

```

describe('ProductCard', () => {
  let fixture: ComponentFixture<ProductCard>;
  let component: ProductCard;

  beforeEach(() => {
    fixture = TestBed.createComponent(ProductCard);
    component = fixture.componentInstance;
  });

  it('renders product name', () => {
    component.product = { id: 1, name: 'Pen', price: 20 };
    fixture.detectChanges();
    expect(fixture.nativeElement.textContent).toContain('Pen');
  });

  it('emits buy event with product', () => {
    const product = { id: 1, name: 'Pen', price: 20 };
    component.product = product;
    fixture.detectChanges();

    let emittedValue: Product | undefined;
    component.buy.subscribe(p => emittedValue = p);

    fixture.debugElement.query(By.css('button.buy')).nativeElement.click();
    expect(emittedValue).toEqual(product);
  });
});

```

```
});  
});
```

5. Testing Services

Services without dependencies — easy:

```
describe('FormatService', () => {  
  let service: FormatService;  
  beforeEach(() => { service = new FormatService(); });  
  
  it('formats currency', () => {  
    expect(service.formatCurrency(1500)).toContain('1,500');  
  });  
});
```

With HTTP — `HttpTestingController` :

```
import { TestBed } from '@angular/core/testing';  
import { provideHttpClient } from '@angular/common/http';  
import { provideHttpClientTesting, HttpTestingController } from '@angular/common/http/testing';  
  
describe('UserService', () => {  
  let service: UserService;  
  let httpMock: HttpTestingController;  
  
  beforeEach(() => {  
    TestBed.configureTestingModule({  
      providers: [provideHttpClient(), provideHttpClientTesting()]  
    });  
    service = TestBed.inject(UserService);  
    httpMock = TestBed.inject(HttpTestingController);  
  });  
  
  it('fetches users', () => {  
    const expected = [{ id: 1, name: 'Ravi' }];  
  
    service.list().subscribe(users => {  
      expect(users).toEqual(expected);  
    });  
  });  
});
```

```

});

const req = httpMock.expectOne('/api/users');
expect(req.request.method).toBe('GET');
req.flush(expected);

httpMock.verify();
});
});

```

Pattern:

- `expectOne(url)` — assert request was made
- `req.flush(data)` — respond
- `httpMock.verify()` — assert no unexpected requests

Important: **no real network calls** in unit tests.

6. Mocking Dependencies

```

describe('UserList', () => {
  let component: UserList;
  let userServiceSpy: jasmine.SpyObj<UserService>;

  beforeEach(() => {
    userServiceSpy = jasmine.createSpyObj('UserService', ['list']);

    TestBed.configureTestingModule({
      imports: [UserList],
      providers: [{ provide: UserService, useValue: userServiceSpy }]
    });

    const fixture = TestBed.createComponent(UserList);
    component = fixture.componentInstance;
  });

  it('loads users on init', () => {
    const users = [{ id: 1, name: 'Ravi' }];
    userServiceSpy.list.and.returnValue(of(users));
  });

```



```

component.ngOnInit();

expect(userServiceSpy.list).toHaveBeenCalled();
expect(component.users).toEqual(users);
});
});

```

- `jasmine.createSpyObj` — fake with named methods
- `.and.returnValue(of(...))` — fake response
- `toHaveBeenCalled()` , `toHaveBeenCalledWith(args)` — verify

7. Testing Async — `fakeAsync` / `tick`

```

import { fakeAsync, tick } from '@angular/core/testing';

it('debounces search', fakeAsync(() => {
  component.search('react');
  tick(299);
  expect(component.results.length).toBe(0);

  tick(1); // total 300ms
  expect(component.results.length).toBeGreaterThan(0);
}));

```

`fakeAsync` controls time. `tick(ms)` advances clock. No real waiting.

8. What to Test as a Junior

Focus on:

1. **Pure business logic** — services without DOM. Easy to write, valuable.
2. **Component output** — given inputs, what does template show?
3. **Event handlers** — clicking calls right method, emits right output.

Skip:

1. **Angular itself** — `@Input` works; testing it is testing Angular.
2. **Trivial getters** — `get fullName()` .
3. **CSS** — visual regression has better tools.

9. End-to-End (E2E)

For full flow tests (login + navigate + create + verify), use **Cypress** or **Playwright**. Run actual browser, click around. Not Karma/Jasmine.

Junior: know they exist. Won't write many.

Common Mistakes

1. **Forgetting `detectChanges()`** — UI doesn't update.
2. **Testing real HTTP** — flaky, slow.
3. **Reading template too early** — before `detectChanges` .
4. **Async without `fakeAsync` or `done`** .
5. **No `httpMock.verify()`** — silent leaks of expectations.
6. **Testing implementation details** — couples test to refactors.
7. **One big test** — break into focused `it` s.

Interview Soundbites

Q: How do you test Angular?

"Unit tests with Jasmine/Karma (or Jest). Set up via `TestBed.configureTestingModule` , render via `TestBed.createComponent` . Trigger via `fixture.detectChanges()` . Mock services with `jasmine.createSpyObj` or fakes. Mock HTTP with `provideHttpClientTesting` + `HttpTestingController` ."

Q: What do you actually test?

"Pure logic — services, utilities, validators. Component output for given inputs. Key event handlers. Skip framework behavior. For end-to-end flows, Cypress or Playwright."

Q: Mock HTTP?

" `provideHttpClientTesting` swaps real backend for in-memory. Inject `HttpTestingController` , use `httpMock.expectOne(url)` to assert request, `req.flush(data)` to respond. `httpMock.verify()` at end."

Mental Model

Five sentences:

1. **describe** groups, **it** tests, **expect().toBe(...)** asserts.

2. `TestBed.configureTestingModule({...}).createComponent(X)` mounts; `fixture.detectChanges()` after every change.
 3. HTTP: `provideHttpClientTesting()` + `HttpTestingController` — `expectOne` / `flush` / `verify` pattern.
 4. Mock services with `jasmine.createSpyObj` ; provide via `{ provide: X, useValue: spy }` .
 5. Focus on business logic, public API, and component contracts; skip framework internals.
-

Topic 31 — State Management

State = data app holds at any moment. Four kinds:

Type	Examples
Server state	API data
Client/UI state	Modal open, sidebar collapsed
Form state	What user typed
URL state	Current route, query params

Most teams confuse them. Strategy depends on type.

1. Levels of State Management

Level 1 — Component-local: Variables on component. Use for state nothing else needs.

Level 2 — Service + signals/BehaviorSubject: Shared service, multiple components see same state. Where most modern Angular apps live.

Level 3 — NgRx / SignalStore: Redux-style store. For very large apps with complex cross-feature state.

Rule: Use the lowest level that solves the problem. Don't reach for NgRx prematurely.

2. Level 1 — Component State

```

@Component({ /* ... */ })
export class TodoList {
  todos = signal<Todo[]>([]);
  filter = signal<'all' | 'active' | 'done'>('all');

  filtered = computed(() => {
    const f = this.filter();
    return this.todos().filter(t => f === 'all' || (f === 'done' ? t.done : !t.done));
  });

  add(text: string) {
    this.todos.update(list => [...list, { id: Date.now(), text, done: false }]);
  }
}

```

Self-contained. No store. Perfect when scope is just this component.

3. Level 2 — Service with Signals

```

@Injectable({ providedIn: 'root' })
export class TodoStore {
  private _todos = signal<Todo[]>([]);
  private _filter = signal<'all' | 'active' | 'done'>('all');

  todos = this._todos.asReadonly();
  filter = this._filter.asReadonly();

  filtered = computed(() => {
    const f = this._filter();
    return this._todos().filter(t => f === 'all' || (f === 'done' ? t.done : !t.done));
  });

  count = computed(() => this._todos().length);
  activeCount = computed(() => this._todos().filter(t => !t.done).length);

  add(text: string) {
    this._todos.update(list => [...list, { id: Date.now(), text, done: false }]);
  }
}

```

```

toggle(id: number) {
  this._todos.update(list =>
    list.map(t => t.id === id ? { ...t, done: !t.done } : t)
  );
}

setFilter(f: 'all' | 'active' | 'done') {
  this._filter.set(f);
}
}

```

Any component injects:

```

@Component({ /* ... */ })
export class TodoList {
  store = inject(TodoStore);
}

```

```

@for (todo of store.filtered(); track todo.id) {
  <div (click)="store.toggle(todo.id)">{{ todo.text }}</div>
}
<p>{{ store.activeCount() }} active</p>

```

Three rules of this pattern:

1. **Private writable signals, public readonly.** Forces consumers to use methods.
2. **All mutations through methods** — validation/logging in one place.
3. **Computed for derived data** — `count` , `activeCount` , `filtered` .

80-90% of real apps live happily at this level.

4. Level 2.5 — Service with BehaviorSubject (Pre-Signal)

Same idea, RxJS-based:

```

@Injectable({ providedIn: 'root' })
export class TodoStore {
  private _todos$ = new BehaviorSubject<Todo[]>([]);
  todos$ = this._todos$.asObservable();
}

```

```
add(todo: Todo) {  
  this._todos$.next([...this._todos$.value, todo]);  
}  
}
```

Same architecture as signals, just Observables. Pre-Angular 16 standard.

5. Level 3 — NgRx (Redux Pattern)

For apps where:

- Many features share complex state
- You need time-travel debugging
- Optimistic UI with rollback
- Multiple devs need predictable patterns

Concepts:

- **State** — single immutable tree
- **Actions** — objects describing what happened
- **Reducers** — pure functions producing new state
- **Selectors** — derived data
- **Effects** — side effects (HTTP, navigation)

```
// Actions  
export const loadTodos = createAction('[Todo] Load');  
export const loadTodosSuccess = createAction('[Todo] Load Success', props<{ todos: To  
export const addTodo = createAction('[Todo] Add', props<{ text: string }>());  
  
// Reducer  
export const todoReducer = createReducer(  
  initialState,  
  on(loadTodosSuccess, (state, { todos }) => ({ ...state, todos, loading: false })),  
  on(addTodo, (state, { text }) => ({  
    ...state,  
    todos: [...state.todos, { id: Date.now(), text, done: false }]  
  })))  
);  
  
// Effect
```

```

loadTodos$ = createEffect(() =>
  this.actions$.pipe(
    ofType(loadTodos),
    switchMap(() => this.todoService.getAll().pipe(
      map(todos => loadTodosSuccess({ todos })),
      catchError(err => of(loadTodosFailure({ error: err.message })))
    ))
  )
);

// Selector
export const selectActiveTodos = createSelector(
  selectAllTodos,
  todos => todos.filter(t => !t.done)
);

// Component
@Component({ /* ... */ })
export class TodoList {
  private store = inject(Store);
  todos$ = this.store.select(selectActiveTodos);

  ngOnInit() { this.store.dispatch(loadTodos()); }
  add(text: string) { this.store.dispatch(addTodo({ text })); }
}

```

For a junior, this is a lot. If you're at a place using NgRx, learn it on the job. For interviews, know the words and concept.

6. NgRx SignalStore (New & Lighter)

Modern alternative — less boilerplate, signal-based:

```

import { signalStore, withState, withMethods, withComputed, patchState } from '@ngrx/

export const TodoStore = signalStore(
  { providedIn: 'root' },
  withState<{ todos: Todo[]; filter: 'all' | 'active' | 'done' }>({
    todos: [],
    filter: 'all'
  })
);

```

```

    }},
    withComputed(state => ({
      activeCount: computed(() => state.todos().filter(t => !t.done).length)
    })),
    withMethods(store => ({
      add(text: string) {
        patchState(store, state => ({
          todos: [...state.todos, { id: Date.now(), text, done: false }]
        }));
      },
      toggle(id: number) {
        patchState(store, state => ({
          todos: state.todos.map(t => t.id === id ? { ...t, done: !t.done } : t)
        }));
      }
    }))
  );

```

Use like a service:

```

store = inject(TodoStore);
this.store.add('Buy milk');
this.store.activeCount();

```

Modern preference when Redux pattern fits but classic NgRx feels heavy.

7. When You Actually Need Level 3

Honest tests:

- More than 2 features sharing complex state?
- Need to inspect/replay state transitions for debugging?
- Optimistic updates with rollback on failure?
- Team big enough that “where does state live?” creates conflict?

Yes to 2+ → consider NgRx (or SignalStore). Most apps: no.

8. Server State Is Different

API data isn’t “client state” — it’s a **cache** of remote state. Different concerns:

- Stale-while-revalidate
- Cache invalidation
- Refetch on focus
- Deduping concurrent requests

In React, `TanStack Query`. Angular doesn't have an officially blessed equivalent — most teams write thin wrappers around `HttpClient` + `signals/RxJS`.

Pattern that works:

```
@Injectable({ providedIn: 'root' })
export class UserApi {
  private http = inject(HttpClient);
  private cache = new Map<number, Observable<User>>();

  getByid(id: number) {
    if (!this.cache.has(id)) {
      this.cache.set(id, this.http.get<User>(`/api/users/${id}`).pipe(shareReplay(1)));
    }
    return this.cache.get(id)!;
  }

  invalidate(id?: number) {
    if (id) this.cache.delete(id);
    else this.cache.clear();
  }
}
```

Distinguish “this is server state, cached” from “this is client/UI state.”

9. Choosing for Your Project

Decision flow:

1. **One component owns it?** component-local signals.
2. **Multiple components share it?** service + signals (`providedIn: 'root'`).
3. **Complex with many cross-feature flows?** `SignalStore` or `NgRx`.
4. **It's API data?** service that fetches + caches.

Try lowest level first. **Don't pre-optimize for “we might need NgRx someday.”**

Common Mistakes

1. **Reaching for NgRx too early** — boilerplate without benefit.
2. **Component state everywhere** — duplicated, drifts out of sync.
3. **Mixing approaches inconsistently** — service here, NgRx there, fields elsewhere.
4. **Mutating store state** — breaks signals + breaks Redux.
5. **Treating server state as client state** — no refetch, stale forever.
6. **Storing derived data instead of computing** — drifts. Use `computed()` .
7. **Big stores** — split by feature.

Interview Soundbites

Q: How manage state in Angular?

"Depends on scope. Component state for self-contained UI — signals or class properties. Shared state — service with `providedIn: 'root'` , exposes signals (or BehaviorSubject in older code). For very large apps with complex cross-feature workflows — NgRx (Redux pattern), or the newer NgRx SignalStore for less boilerplate. Use lowest level that works."

Q: When use NgRx?

"When multiple features share complex state, you need predictable patterns across a large team, want time-travel debugging, or have optimistic UI flows. Otherwise — service with signals usually enough. Most apps don't need it."

Q: Signals vs RxJS for state?

"Signals for synchronous reactive state — UI flags, computed derived data, app data after fetch. RxJS for async streams over time — HTTP responses, user input streams, WebSockets. Modern code uses signals for state, RxJS for events; bridge with `toSignal` and `toObservable` ."

Q: Server state vs client state?

"Server state is cached remote data — needs caching, refetch, invalidation, dedup. Client state is genuinely local — UI flags, form drafts, navigation. Treating them the same makes both worse — server state grows stale, client state grows over-architected."

Mental Model

Five sentences:

1. **Four kinds of state — server, client/UI, form, URL. Treat them differently.**

2. **Component-local** **service + signals** **SignalStore/NgRx**. Pick lowest that works.
 3. **Service pattern**: private writable signals, public readonly, mutations via methods, `computed()` for derived.
 4. **Server state is a cache, not “client state”** — needs invalidation/refetch strategy.
 5. **Reach for NgRx when scale demands it, not preemptively.**
-

Topic 32: Performance Optimization

The big-picture lessons: separate load from runtime, measure first, fix in priority order.

1. The Two Kinds of Performance

Type	What it means	Tools to fix
Load performance	Time until something useful renders	Bundle splitting, lazy loading, SSR, images
Runtime performance	Smoothness after load	Change detection, virtual scrolling, debouncing, OnPush

Treat as separate problems. 4-second initial load = load. Laggy scroll on 1000-item list = runtime.

2. Measure First

Don't optimize blindly. Profile, identify, fix, re-profile.

Chrome DevTools Performance tab

Runtime CPU/render/paint profile. Record action stop look at flame chart for long red bars.

Angular DevTools (browser extension)

Angular-specific. Shows component tree and which components ran CD per cycle and for how long. **Critical for diagnosing CD problems.**

Bundle analyzer

```
ng build --stats-json
npx webpack-bundle-analyzer dist/your-app/stats.json
```

Reveals accidentally-imported heavy libraries.

3. Load Performance — Big Levers (in impact order)

A. Lazy load everything off the critical path

Topic 21. Eager: home/login/navbar. Lazy: admin/settings/dashboards. Biggest single win.

B. @defer for below-the-fold

Comments, chart widgets, social embeds `on viewport` / `on idle` .

C. Audit bundle

Bundle analyzer often reveals:

- `moment.js` (290 KB) switch to `Intl.DateTimeFormat`
- Full `lodash` import individual functions
- Duplicate libraries
- Unused stuff still in `package.json`

30 minutes of bundle auditing can shave 200+ KB.

D. Production builds

```
ng build # production (default in Angular 12+)
```

Enables AOT, minification, tree-shaking, dead code elimination, optimized CD. **Always test against `ng build` , not `ng serve` .**

E. Server-Side Rendering

`ng add @angular/ssr` . Faster First Contentful Paint, deployment-architecture decision.
Junior level: know it exists.

F. Image optimization

```
import { NgOptimizedImage } from '@angular/common';

@Component({
  imports: [NgOptimizedImage],
  template: `<img ngSrc="hero.jpg" width="800" height="400" priority />`
})
```

Adds lazy loading, prevents layout shift, prioritizes above-fold.

G. Preload critical lazy chunks

```
withPreloading(PreloadAllModules) .
```

4. Runtime Performance — Big Levers

A. Change detection running too often

Default CD checks every component on every event. Fixes:

- **OnPush** on components that don't need to react to everything.
- **Signals** — Angular knows exactly what to re-check.
- **Stop calling functions in templates.** `{{ getTotal() }}` runs every cycle. Use `computed()` or pipe.
- **NgZone.runOutsideAngular(...)** for tight loops (analytics, animations).

In Angular DevTools, components re-running every cycle are OnPush candidates.

B. Rendering long lists

1000 items = 1000 DOM nodes. Even with OnPush, lots of bindings.

Fixes:

- **track in @for** — use stable ID (`track item.id`).
- **Virtual scrolling** — render only visible:

```
import { ScrollingModule } from '@angular/cdk/scrolling';

@Component({
```

```

imports: [ScrollingModule],
template: `
  <cdk-virtual-scroll-viewport itemSize="50" class="viewport">
    <div *cdkVirtualFor="let item of items" class="item">{{ item.name }}</div>
  </cdk-virtual-scroll-viewport>
`,
styles: [`.viewport { height: 400px; overflow: auto; } .item { height: 50px; }`]
})

```

100,000 items scroll smoothly — only ~10-20 in DOM at any time.

Threshold: ~100 items.

C. Expensive computations every change

- **computed() signals** — memoize derived state.
- **Pure pipes** — same for template transformations.
- **Debounce** expensive triggers (`debounceTime(300)` on filter inputs).

D. Subscriptions and memory leaks

Forgotten subscriptions = component never freed. Slowdowns over time.

- **async pipe** in templates.
- **toSignal()** — same benefit, signal-flavored.
- **takeUntilDestroyed()** for manual subscriptions.

#1 cause of “starts fast, gets slow.”

E. Synchronous heavy work

- Defer non-critical work to `setTimeout(..., 0)` or `requestIdleCallback`.
- For heavy computation: Web Workers (`ng g web-worker`).

5. The OnPush + Signals Recipe

```

@Component({
  selector: 'app-product-list',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <h3>{{ products().length }} products — total  {{ total() }}</h3>

```

```

    @for (p of products(); track p.id) {
      <app-product-card [product]="p" />
    }
  ,
})
export class ProductList {
  products = input.required<Product[]>();
  total = computed(() => this.products().reduce((sum, p) => sum + p.price, 0));
}

```

- Component re-checks only when `products` input reference changes or `total` changes
- Total is memoized
- `track p.id` reuses DOM nodes

Scale this pattern CD footprint shrinks dramatically.

Mantra: OnPush + signals + immutable updates + `track` + `async` pipe.

6. Anti-Patterns

Methods in templates

```
<p>{{ getFormattedDate(user.createdAt) }}</p> <!-- every cycle -->
```

Use pipe (`{{ user.createdAt | date }}`) or `computed()` .

Heavy getters

```
get formattedTotal() { return this.items.reduce(...).toFixed(2); }
```

Same problem. Use `computed()` .

Mutating arrays/objects

```
this.items.push(newItem); // breaks OnPush, pure pipes, track
```

Immutable: `this.items = [...this.items, newItem]` .

ngStyle / ngClass unnecessarily

```
<div [ngStyle]="getStyles()"> <!-- recomputes every cycle -->
```

Use `[class.x]` / `[style.x]` .

Manual subscriptions

Switch to `async` pipe or `toSignal()` .

Whole-library imports

```
import _ from 'lodash';           // imports everything
import { debounce } from 'lodash'; // tree-shakeable
```

Heavy widgets in eager bundle

Chart libraries, WYSIWYG, video players `@defer` or lazy-load that page.

7. The Junior Checklist

- Production build for testing. Always.
- Lazy-load anything not on critical path.
- `@defer` below-the-fold (comments, charts, embeds).
- `OnPush` on components taking inputs without unpredictable internal state.
- Signals for state — `computed()` for derived.
- `track` on every `@for` — stable ID, not `$index` .
- Virtual scrolling for >100 items.
- `async` pipe or `toSignal()` for template Observables.
- No method calls in templates for anything non-trivial.
- Immutable updates for arrays/objects.
- `NgOptimizedImage` for hero/gallery images.
- Bundle audit once.
- Preload lazy chunks with `withPreloading(PreloadAllModules)` .

8. A Real Optimization Story

Initial: Loads in 6s. Customer list (500 items) scrolls laggy.

Round 1 — load: Bundle analyzer finds moment.js (290 KB), unused chart library (180 KB), duplicate rxjs. Fix imports, drop moment. 3.2s.

Round 2 — lazy: Bundle still 1.4 MB. Admin panel ships in main bundle. Move to `loadChildren` . 600 KB initial. 1.8s.

Round 3 — virtual scroll: List laggy. Add `cdk-virtual-scroll-viewport` . Smooth even at 5000 items.

Round 4 — CD: Angular DevTools shows 30-component tree re-checking on every keystroke. Apply `OnPush`. State signals. CD overhead ~80% lower.

Round 5 — long task: Profile shows 300ms blocking task filtering. Trace `getCustomers()` method called in template. `computed()` .

Result: 1.8s load, smooth, no jank. Each round measured before/after.

Common Mistakes

1. **Optimizing without profiling.**
2. **Comparing dev to production.**
3. **`track $index` everywhere** — every reorder rebuilds DOM.
4. **`OnPush` on components that mutate internally** — stale UI.
5. **Virtual scrolling for short lists.**
6. **Lazy-loading the home page.**
7. **Not measuring after each fix.**
8. **One-time optimization pass** — regressions happen on every feature.
9. **Spending hours saving 5 KB.**
10. **Ignoring time in templates.**

Interview Soundbites

Q: Make an Angular app fast?

“Separate load and runtime. Load: lazy-load non-critical routes with `loadChildren` , preload after initial paint, `@defer` for below-the-fold, audit bundle, production builds. Runtime: `OnPush` + signals or `async` pipe, stable `track` on `@for` , virtual scrolling for long lists, no method calls in templates, immutable updates so `OnPush` detects.”

Q: How find what's slow?

“Three tools. Angular DevTools shows which components ran CD per cycle. Chrome

DevTools Performance tab shows long tasks and flame chart. For load — `ng build --stats-json` + bundle analyzer. Measure first, fix biggest, re-measure.”

Q: What does OnPush do?

“Limits when Angular checks component. Re-checks only when `@Input` reference changes, event fires inside, async pipe emits, signal it reads changes. Catch: requires immutable updates — mutating array in place doesn’t trigger.”

Q: Virtual scrolling?

“Only render visible rows. As user scrolls, Angular adds entering and removes leaving, recycling DOM. 10000 items might have ~20 nodes in DOM. CDK’s `<cdk-virtual-scroll-viewport>` handles it — give fixed `itemSize` and viewport height.”

Q: Debug slow scroll?

“Chrome Performance, Record, scroll, stop. Long red bars on main thread = blocking. Look at call stack — usually function call in template or Observable emitting too frequently. Fixes: virtualize, `computed()` / pipe, debounce.”

Q: When NOT use signals?

“Signals for synchronous reactive state. For async streams over time — HTTP, user input streams, WebSockets — RxJS Observables still right tool because of operator library. Most apps use both, bridged with `toSignal()` / `toObservable()` .”

Mental Model

Seven sentences:

1. **Performance is two problems — load and runtime.**
 2. **Always measure first. Angular DevTools + Chrome Performance + bundle analyzer cover 95%.**
 3. **Load: lazy-load critical path off, audit bundles, optimize images, preload after.**
 4. **Runtime: OnPush + signals + immutable + `track` + `async` pipe + virtual scrolling. Compound effects.**
 5. **Function calls in templates are silent killers — replace with pipes/ `computed()` .**
 6. **Forgotten subscriptions cause “starts fast, gets slow.”**
 7. **Optimize what’s slow. Profile, fix, verify, repeat. Not “apply every optimization upfront.”**
-

Topic 33: Deployment & Build Optimization

How Angular apps actually get shipped. Smaller, practical topic — mostly one-time setup.

1. What `ng build` Produces

```
ng build
```

```
dist/  
  my-app/  
    browser/  
      index.html      HTML shell  
      main-XXXX.js    app code + Angular  
      polyfills-XXXX.js browser compat  
      styles-XXXX.css  bundled CSS  
      chunk-XXXX.js    lazy-loaded chunks  
      assets/          static files
```

File	Purpose
index.html	Just <code><app-root></code> + <code><script></code> tags
main-*.js	Code + Angular
polyfills-*.js	Browser compat (small now)
chunk-*.js	Each lazy route
styles-*.css	Bundled, minified
assets/	Static files copied as-is

The `XXXX` is a **content hash** — when file content changes, hash changes, browser cache busts. Automatic.

Deploy that entire `dist/my-app/browser/` folder. Just static files.

2. Build Configurations — Prod vs Dev

```
ng build           # production (default in v17+)
ng build --configuration=development
ng serve           # dev with hot reload
ng serve --configuration=production
```

Production enables:

- **AOT compilation** — templates compile at build time
- **Minification** — names shortened
- **Tree-shaking** — drop unused
- **Dead code elimination**
- **Bundling**
- **Content hashes**
- **External source maps**

Development prioritizes build speed and debuggability — less optimization, inline source maps, named exports preserved.

Always test against `ng build` . Dev builds 5-10x slower, 2-4x bigger. Performance numbers from dev are meaningless.

3. Environment Configuration

File-based (classic)

```
// src/environments/environment.ts
export const environment = {
  production: false,
  apiUrl: 'http://localhost:3000/api'
};

// src/environments/environment.prod.ts
export const environment = {
  production: true,
  apiUrl: 'https://api.example.com'
};
```

angular.json :

```
"configurations": {  
  "production": {  
    "fileReplacements": [  
      { "replace": "src/environments/environment.ts", "with": "src/environments/environm  
    ]  
  }  
}
```

Usage:

```
import { environment } from '../environments/environment';  
  
@Injectable({ providedIn: 'root' })  
export class ApiService {  
  private base = environment.apiUrl;  
}
```

`ng build` substitutes the prod file. Each environment gets own bundle.

Runtime config (modern alternative)

```
export const API_URL = new InjectionToken<string>('API_URL');  
  
providers: [  
  { provide: API_URL, useValue: window.__env__?.apiUrl ?? '/api' }  
]
```

Populate `window.__env__` from `env.js` loaded before Angular bundle. Same artifact, many deploys.

4. The SPA Fallback — `/users/42` 404 Problem

Subtle gotcha everyone hits.

Your app is single page (`index.html`). Routes like `/users/42` handled by Angular Router in browser. Server never sees them as real pages.

User on `/users/42` hits refresh. Browser asks server for `/users/42`. Server has no file. **404.**

Fix: configure server to **fall back to `index.html` for any unmatched route.**

Host	Configuration
Nginx	<code>try_files \$uri \$uri/ /index.html;</code>
Apache	<code>.htaccess</code> with <code>RewriteRule ^ index.html [L]</code>
Vercel / Netlify	Auto-detected, or <code>_redirects</code> file with <code>/* /index.html 200</code>
AWS S3 + CloudFront	404 <code>/index.html</code> with 200 in error pages
Firebase Hosting	<code>"rewrites": [{ "source": "**", "destination": "/index.html" }]</code>
GitHub Pages	<code>404.html</code> workaround or hash routing

Fallback says “if you don’t know this URL, give `index.html` anyway.” Angular boots in browser and routes.

Alternative — hash routing (`/#/users/42`): everything after `#` client-side, server never sees it. Avoids problem but uglier URLs.

```
provideRouter(routes, withHashLocation())
```

5. Common Deployment Targets

A. Static file host (Vercel, Netlify, Firebase, GitHub Pages, S3)

Simplest. Build, upload `dist/my-app/browser/`, configure SPA fallback. Done.

- **Vercel/Netlify:** Connect Git, build `ng build`, output `dist/my-app/browser`. Auto-deploys.
- **Firebase:** `firebase-tools`, `firebase init hosting`, `firebase deploy`.
- **GitHub Pages:** `ng add angular-cli-ghpages` then `ng deploy`. Free; use `withHashLocation()` to avoid 404s.

B. Cloud storage + CDN (S3 + CloudFront, GCS + Cloud CDN)

Same idea, pieces yourself. More work, more control, often cheaper at scale.

C. Docker container

```
# Build stage
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Serve stage
FROM nginx:alpine
COPY --from=build /app/dist/my-app/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

nginx.conf with SPA fallback:

```
server {
  listen 80;
  root /usr/share/nginx/html;
  location / {
    try_files $uri $uri/ /index.html;
  }
}
```

Deploy to Kubernetes, ECS, Cloud Run, fly.io.

D. Backend-bundled

Serve Angular as static from existing backend:

```
// Express
app.use(express.static('dist/my-app/browser'));
app.get('*', (_, res) => res.sendFile(path.join(__dirname, 'dist/my-app/browser/index.htm
```

E. SSR

SEO, faster FCP, link previews. `ng add @angular/ssr` . Adds Node server, server bundle, hydration wiring. Deploy where Node runs (Vercel auto-detects, Cloud Run, VPS).

Junior probably won't set up. Know:

- Deployment architecture decision
- Requires Node hosting
- Wrap browser APIs in `isPlatformBrowser()` checks

6. Built-In Optimizations

Optimization	What it does	When you'd touch
AOT	Templates compile at build	Always on; can't disable in v15+
Tree-shaking	Drop unused exports	Always on
Code splitting	Lazy routes separate chunks	Configure via <code>loadChildren</code> / <code>loadComponent</code>
Minification	Short names, no whitespace	Always on in prod
CSS optimization	Minify, autoprefix	Always on in prod
Hashed filenames	Content hash for caching	Always on in prod
Lazy CSS	CSS for lazy chunks loads with chunk	Auto
Build budgets	Error if bundle exceeds limit	Configure in <code>angular.json</code>

Build budgets

```
"budgets": [  
  { "type": "initial", "maximumWarning": "500kb", "maximumError": "1mb" },  
  { "type": "anyComponentStyle", "maximumWarning": "2kb", "maximumError": "4kb" }  
]
```


Over 500 KB initial warning. Over 1 MB build fails. Catches “someone added enormous dependency” before shipping. Always keep on.

7. CI/CD Pipeline

```
# .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm test -- --watch=false --browsers=ChromeHeadless
      - run: npm run build
      - name: Deploy
        run: |
          # Deploy `dist/my-app/browser` to your host
```

Pattern: **install test build deploy**. Failed tests block deploy.

8. Pre-Deployment Checklist

- **Build for production.** `ng build` , never dev.
- **Test prod build locally.** `npm http-server dist/my-app/browser -p 8080` . Catch routing/SSR bugs.
- **Verify environment config.** Right API URLs, feature flags.
- **Check bundle sizes.** Anything surprising? Run bundle analyzer.
- **Verify SPA fallback.** Visit `/users/42` directly. Should load, not 404.
- **No hardcoded localhost or dev secrets.**

- **HTTPS everywhere.** Mixed content breaks in production.
- **Service worker check** if you have one.
- **Accessibility (Lighthouse/axe).**
- **Lighthouse Performance ≥ 80** target.

9. Common Mistakes

1. **Deploying dev build.** 5x worse performance, 4x bigger.
2. **Forgetting SPA fallback.** Deep URLs 404 on refresh.
3. **Hardcoding API URLs.**
4. **Skipping local prod test.** AOT can surface template errors not in `ng serve`.
5. **Ignoring bundle budgets.** Careless `import * as _ from 'lodash'` adds 70 KB.
6. **Caching `index.html`.** Stale apps after deploy. Cache hashed assets for a year; `index.html` for 0s or `must-revalidate`.
7. **Mixing HTTP and HTTPS.** Browsers block as mixed content.
8. **Public source maps to production.** Exposes code. Ship to authenticated error tracking (Sentry) or skip.
9. **Not testing build in CI.** “Works on my machine” CI break.
10. **Treating build as black box.**

10. Interview Soundbites

Q: How deploy an Angular app?

“`ng build` static bundle in `dist/my-app/browser/`. Any static host: Vercel, Netlify, Firebase, S3+CloudFront, `nginx` in Docker. Always: SPA fallback — server serves `index.html` for unmatched routes, else deep links 404.”

Q: Dev vs prod build?

“Prod: AOT, minification, tree-shaking, dead-code elimination, hashed filenames. Dev: prioritizes fast rebuilds and debugging. Often 5x performance difference — always measure against `ng build`.”

Q: Different environments?

“Classic: file-based environments — `environment.ts` for dev, `environment.prod.ts` for prod, `fileReplacements` in `angular.json` swap at build time. Modern: runtime config from small JSON or `window.__env__`, via `InjectionToken`. Same artifact, many deploys.”

Q: Why does refreshing `/users/42` 404?

“Angular router runs in browser. Server has no file at `/users/42` — just `index.html`

and JS. Refresh asks server directly. Fix: configure host to fall back to `index.html` for any unmatched route. Angular boots, sees URL, routes.”

Q: Typical bundle size?

“Small-medium app: initial bundle 200-500 KB gzipped after prod build, plus lazy chunks. Budget warning around 500 KB to catch regressions. Biggest savings: lazy-load non-critical routes, optimize images, audit accidentally-imported heavy libs, `@defer` for below-the-fold.”

Q: Cache-busting?

“`ng build` includes content hash in filename — `main-A1B2C3D4.js`. Content changes hash changes forces re-download. `index.html` references hashed names but isn't hashed itself. Configure aggressive caching for hashed assets (year+), short caching for `index.html`. Deploy: `index.html` revalidates, sees new hashed names, downloads new bundles.”

Mental Model

Five sentences:

1. `ng build` static folder `dist/my-app/browser/`. Upload to any static host.
 2. Configure SPA fallback — server serves `index.html` for unmatched routes.
 3. Production builds enable AOT, minification, tree-shaking, code splitting, hashed filenames. Test optimizations against `ng build`, not `ng serve`.
 4. Environment config: `environment.ts` files (swapped at build) or runtime JSON (one artifact, many deploys).
 5. CI/CD: install test build deploy. Bundle budgets catch regressions.
-

Closing — What Next

You've now covered every topic on your list — TypeScript through deployment. Concretely:

You should now be able to:

- Build a non-trivial Angular feature from `ng new` to deployed URL.
- Explain every topic with code-level answers in an interview.
- Read others' Angular code (modern or legacy NgModule) and understand it.
- Debug common issues (CD problems, leaks, missing imports, routing 404s) by going to the right concept.

The honest next step is to build. Knowledge becomes skill only when fingers do the work. Pick a non-trivial idea (notes app with auth, markdown blog, kanban, expense tracker), build end-to-end, ship it.

For your 10-day deadline:

- **Days 1-2:** Build a real thing for the work project. Get it running locally. Don't optimize, don't worry about edge cases. Make components render, data flow, routes work.
- **Days 3-6:** Add real features — services, forms, routing, HTTP. Where reading becomes muscle memory. You'll hit bugs. Good. Debugging is learning.
- **Days 7-8:** Polish. Validation, error handling, loading states. Make it presentable.
- **Day 9:** Mock interview. Out loud. Answer every "Interview Soundbite" in this doc without looking. Stumbles — re-read those topics.
- **Day 10:** Light review. Sleep early. Don't cram.

For the interview:

- What sets you apart: *"I built X using Y, here's the trade-off."* Concrete, not abstract.
- It's okay to say *"Haven't used that yet, but I'd reason about it like..."* — interviewers prefer this to confident wrong answers.
- Likely topics: component communication, services + DI, lifecycle hooks, signals vs Observables, change detection, reactive forms. Re-read those soundbites the night before.

Closing thought. You went from "I don't know JS properly" to working through 33 dense topics. That's a real signal about how you learn. Trust it. The first real Angular bug that takes you 90 minutes to figure out — that's when this stops being knowledge and becomes skill.

Now close this doc and build something.

Good luck.

— End of curriculum —